



CI Amazonia — Capacitacao em Microeletronica

Matheus Serrao Uchoa Matricula: 22052633

RELATORIO TECNICO FINAL

Unidade 7 | Capitulo 5 – Desafio de Design de Circuitos Integrados

Aqua Monitor: Plataforma IoT de Monitoramento da
Qualidade da Agua em Viveiros de Piscicultura
com Projeto de Conversor Analogico-Digital
SAR de 4-Bits em Tecnologia CMOS 180nm

Disciplina: Design de Circuitos Integrados – CI Amazonia

Professor: Thiago Brito

Manaus – AM Maio de 2026

Sumário

1	Introducao	4
1.1	Contexto e Motivacao	4
1.2	O Problema da Qualidade de Aquisicao Analogica no ESP32	4
1.3	Objetivo Final: Integracao do Chip SAR ADC no Aqua Monitor	4
1.4	Escopo e Organizacao deste Relatorio	5
2	Arquitetura Geral do Sistema Aqua Monitor	5
2.1	Fluxo de Dados Completo	7
2.2	Tabela de Componentes do Sistema	8
3	Camada de Hardware: Sensores e Microcontrolador	8
3.1	O Microcontrolador ESP32	8
3.2	Sensor de Temperatura – DS18B20	9
3.3	Sensor de pH – PH4502C	9
3.4	Sensor de Turbidez – SEN0189	10
3.5	Sensor de Condutividade/TDS – Gravity TDS	11
3.6	Montagem Fisica do Sistema	12
4	Camada de Firmware: Software Embarcado no ESP32	13
4.1	Visao Geral do Firmware	13
4.2	Estrutura de Dados de Telemetria	13
4.3	Media de Leituras ADC – Reducao de Ruído	13
4.4	Upload para o Firebase Firestore	14
4.5	Controle Remoto via MQTT	14
5	Camada de Interface: Aplicativo Mobile	15
5.1	Tecnologias Utilizadas	15
5.2	Funcionalidades Implementadas	16
5.3	Backend Inteligente – Chatbot com IA	16
6	Revisao da Literatura	16
6.1	Monitoramento de Qualidade da Agua por IoT	16
6.2	ADC SAR para Aplicacoes IoT de Baixo Consumo	17
6.3	Comparadores Diferenciais CMOS de Baixo Consumo	17
6.4	Layout Common-Centroid e Modelo de Mismatch de Pelgrom	17
6.5	Extracao de Parasitas e Simulacao Pos-Layout	18
7	Modulo de Microeletronica: O SAR ADC Customizado	18
7.1	Por Que um ADC Customizado?	18

7.2	Especificacoes Eletricas	19
7.3	Arquitetura: Os Tres Blocos do SAR ADC	19
8	Fundamentacao Matematica do SAR ADC	21
8.1	Equacao de Transferencia e Algoritmo de Conversao	21
8.2	Orcamento de Ruído	21
8.3	Analise de Descasamento de Capacitores – Modelo de Pelgrom	22
8.4	ENOB e Figura de Merito de Walden	22
9	 dominio Digital: Maquina de Estados SAR em SystemVerilog RTL	22
9.1	O Que e RTL e Por Que Usamos SystemVerilog?	23
9.2	Os 7 Estados da FSM SAR	23
9.3	Codigo SystemVerilog Comentado	23
9.4	Resultados da Simulacao Digital (Icarus Verilog)	26
10	 dominio Analogico I: DAC de Redistribuicao de Carga	27
10.1	O Que e um DAC de Redistribuicao de Carga?	27
10.2	Dimensionamento do Banco Capacitivo	27
10.3	O Netlist SPICE do DAC – Decisoes de Implementacao	27
10.4	Resultados da Simulacao do DAC	28
11	 dominio Analogico II: Comparador Diferencial de Dois Estagios	29
11.1	O Que o Comparador Precisa Fazer?	29
11.2	Topologia: Por Que Par Diferencial PMOS?	30
11.3	Dimensionamento Transistor por Transistor	30
11.4	O Netlist SPICE do Comparador – Anotado	31
11.5	Resultados de Simulacao Pre-Layout – Curva DC	33
11.6	Resultados de Simulacao Pre-Layout – Resposta Transiente	34
11.7	Layout Fisico – Common-Centroid ABBA	34
11.8	Extracao de Parasitas e Simulacao Pos-Layout	36
11.9	Verificacao LVS – Layout vs. Schematic	37
12	Analise de Desempenho e Integracao ao Sistema	38
12.1	Orcamento de Ruído Consolidado	38
12.2	Comparacao com o Estado da Arte	38
12.3	Integracao: Como o ADC Customizado se Encaixa no Aqua Monitor	38
13	Conclusoes e Trabalhos Futuros	39
13.1	O Que Foi Construido	39
13.2	Limitacoes e Trabalhos Futuros	40

Resumo

Este relatorio tecnico descreve o desenvolvimento integral do sistema **Aqua Monitor**, uma plataforma de codigo aberto voltada ao monitoramento continuo da qualidade da agua em viveiros de piscicultura da regio amazonica. O sistema e composto por quatro camadas funcionais integradas: (i) **Hardware embarcado** – microcontrolador ESP32 (sistema em chip da Espressif com Wi-Fi integrado) conectado a quatro sensores (temperatura DS18B20, pH PH4502C, turbidez SEN0189 e solidos dissolvidos Gravity TDS), com comunicacao via protocolo MQTT (*Message Queuing Telemetry Transport*, protocolo padrao para IoT – *Internet of Things*) e envio de dados a plataforma de nuvem Firebase/Firestore; (ii) **Backend inteligente** – servidor Python/FastAPI com agente de linguagem (Ollama qwen2:0.5b) para interpretacao de perguntas em linguagem natural sobre qualidade da agua; (iii) **Aplicativo mobile** – desenvolvido em Ionic/React/TypeScript, com monitoramento em tempo real, historico de dados com graficos, exportacao em PDF e controle remoto do ESP32; (iv) **Modulo de microeletronica** – projeto de um Conversor Analogico-Digital de Aproximacoes Sucessivas (ADC SAR – *Successive Approximation Register Analog-to-Digital Converter*) de 4 bits em tecnologia CMOS (*Complementary Metal-Oxide-Semiconductor*) de 1,8 V (180nm), concebido para substituir o ADC interno do ESP32, cuja qualidade e degradada pelo ruido radiado pelo modulo Wi-Fi integrado. O projeto cobre o fluxo completo de design de circuitos integrados de sinal misto: especificacao eletrica, Maquina de Estados Finita (FSM – *Finite State Machine*) em SystemVerilog RTL (*Register Transfer Level*), DAC (*Digital-to-Analog Converter*) de redistribuicao de carga ($C_0 = 100\text{ fF}$, ruido $kT/C = 1,6\text{ mV}$ com margem de $35\times$ sobre $V_{LSB}/2$), comparador diferencial PMOS de dois estagios ($A_1 = 95\text{ V/V}$, $t_{pd} = 7,4\text{ ns}$ pre-layout), layout fisico Common-Centroid ($4025\text{ }\mu\text{m}^2$), extracao de parasitas e simulacao pos-layout ($\Delta t_{pd} = +2,6\text{ ns}$, $\Delta V_{th} = 0,3\text{ mV}$), alem de verificacao LVS (*Layout vs. Schematic*) sem erros (7 transistores, 10 nos, 0 violacoes DRC – *Design Rule Check*). A FOM (*Figure of Merit*) de Walden calculada e $\text{FOM}_{\text{Walden}} = 45\text{ fJ/conv-step}$.

Palavras-chave: IoT, Aquicultura, Monitoramento Ambiental, SAR ADC, CMOS 180nm, Mixed-Signal, Redistribuicao de Carga, Comparador Diferencial, Common-Centroid, SystemVerilog, LVS.

1 Introducao

1.1 Contexto e Motivacao

A piscicultura e uma das atividades economicas de maior crescimento na regioao Norte do Brasil, com o Amazonas figurando entre os principais estados produtores. A qualidade da agua e o fator critico de sucesso: variacoes de pH, temperatura, turbidez ou condutividade fora das faixas ideais causam estresse nos animais, reducao da imunidade e, em casos extremos, morte em massa de planteis, com prejuizos que podem chegar a dezenas de milhares de reais por ciclo. O monitoramento tradicional baseia-se em coletas manuais realizadas uma ou duas vezes ao dia, o que e insuficiente para detectar eventos rapidos como um pico de amonia ou uma queda brusca de oxigenio dissolvido [13].

O **Aqua Monitor** nasce para resolver esse problema: uma plataforma de monitoramento contínuo, de baixo custo, conectada a nuvem e acessível via aplicativo móvel, que permite ao piscicultor acompanhar os parametros da agua de qualquer lugar, a qualquer hora, e receber alertas imediatos quando valores saem das faixas recomendadas.

1.2 O Problema da Qualidade de Aquisicao Analogica no ESP32

O microcontrolador ESP32 – escolhido pelo custo acessível e pelo modulo Wi-Fi integrado – possui internamente um SAR ADC de 12 bits. Contudo, a Espressif, fabricante do chip, documenta em seu manual tecnico [10] que o modulo de radio Wi-Fi 802.11n introduz **ruido de modo comum de ate 60 mV** no rail de alimentacao analogica (AV_{DD}), degradando a qualidade das leituras. Para o sensor de pH – cuja curva de calibracao tem sensibilidade de apenas $\sim 58\text{mV/pH}$ – um ruido de 60 mV representa mais de 1 pH de incerteza, tornando as leituras inuteis sem filtragem intensiva.

Esta limitacao justifica o projeto de um **ADC customizado dedicado**, projetado especificamente para a faixa dinamica dos sensores do Aqua Monitor, sem a interferencia do transceptor de radio.

1.3 Objetivo Final: Integracao do Chip SAR ADC no Aqua Monitor

O objetivo final deste projeto e a integracao de um chip ADC SAR customizado em CMOS 180nm na cadeia de sinal do Aqua Monitor, substituindo o ADC interno do ESP32 para os canais de pH e turbidez. Como ilustrado na Figura 1, o chip e posicionado entre o sensor analogico e o ESP32, isolando completamente a conversao A/D do ruido eletromagnetico gerado pelo transceptor Wi-Fi.

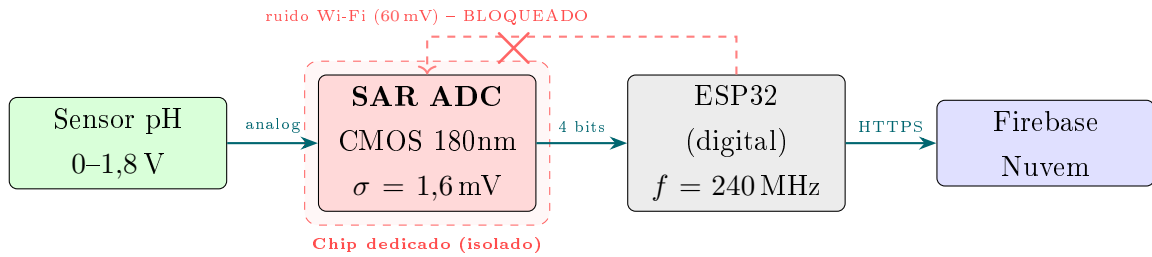


Figura 1: Objetivo de integracao do projeto: o chip SAR ADC CMOS 180nm e inserido entre o sensor e o ESP32, bloqueando o ruído Wi-Fi de 60 mV que degrada o ADC interno do microcontrolador. Com $\sigma_{kT/C} = 1,6 \text{ mV}$, o chip melhora a qualidade de medicao de pH em mais de $30\times$.

1.4 Escopo e Organizacao deste Relatorio

Este relatorio apresenta o Aqua Monitor como um produto completo, descrevendo todas as camadas do sistema com nivel de detalhe suficiente para que qualquer leitor – independente de background tecnico – compreenda o que foi construido, por que cada decisao de design foi tomada, e o que cada componente faz. A organizacao e a seguinte:

- **Secoos 2–4:** Visao geral do sistema e suas camadas de hardware, firmware e nuvem.
- **Secao 5:** Aplicativo mobile – interface do usuario.
- **Secao 6:** Revisao da literatura sobre as tecnologias empregadas.
- **Secoos 7–10: Nucleo de microeletronica** – projeto completo do SAR ADC (foco principal deste trabalho para a disciplina de CI).
- **Secoos 11–12:** Integracao, resultados consolidados e conclusoes.

2 Arquitetura Geral do Sistema Aqua Monitor

O Aqua Monitor e organizado em quatro camadas que se comunicam entre si, conforme o diagrama da Figura 2. Esta secao descreve cada camada em alto nivel; as secoes seguintes aprofundam cada uma.

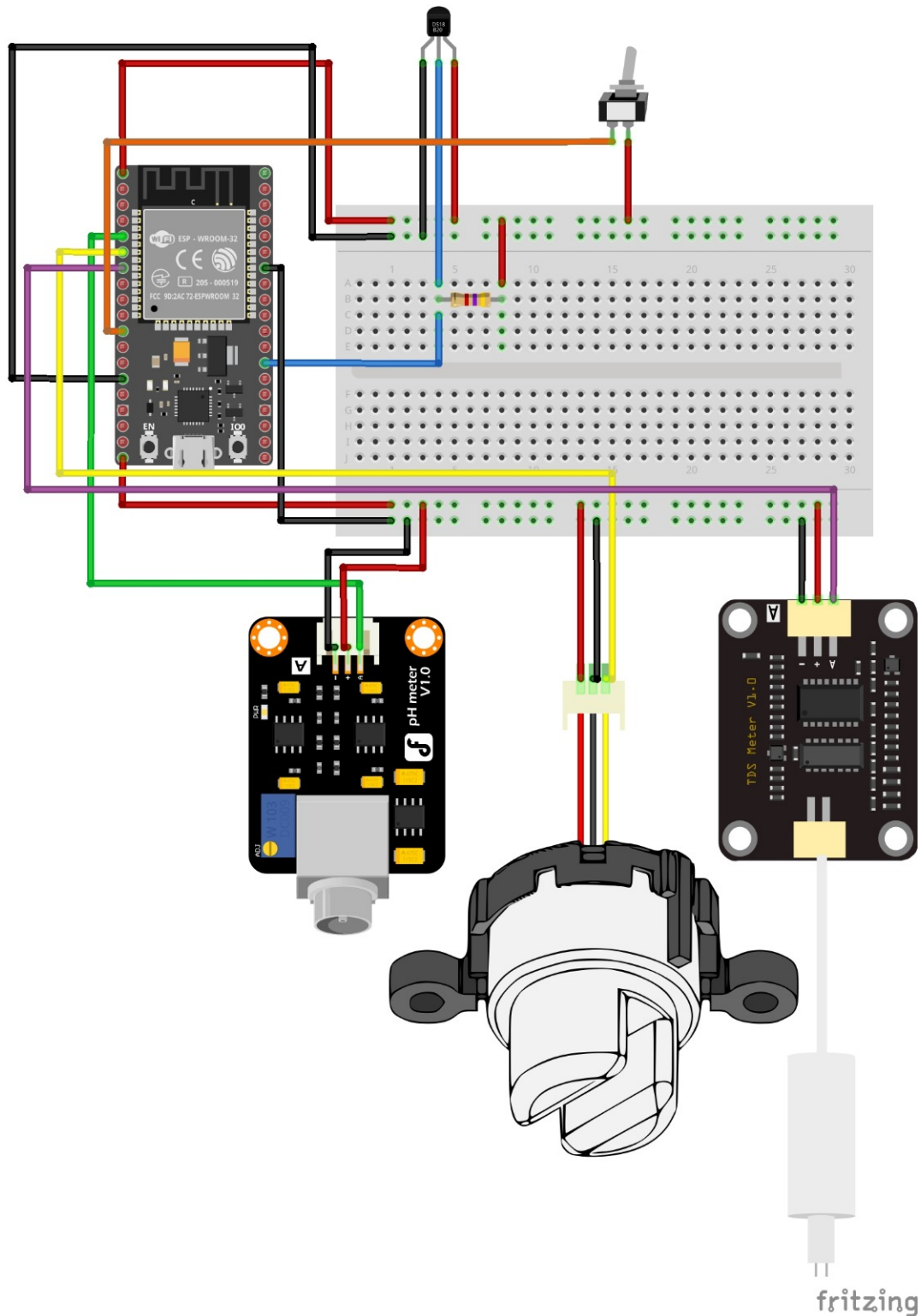


Figura 2: Diagrama do circuito do Aqua Monitor simulado na plataforma Wokwi, exibindo o ESP32 conectado aos quatro modulos de sensores. Da esquerda para a direita: sensor de temperatura DS18B20 (azul, barramento OneWire), modulo de pH PH4502C, sensor de turbidez SEN0189 e modulo Gravity TDS, todos com saida analogica de tensao.

2.1 Fluxo de Dados Completo

A Figura 3 ilustra o fluxo de dados completo do sistema em quatro camadas, desde os sensores fisicos imersos na agua ate a interface visual do usuario no celular.

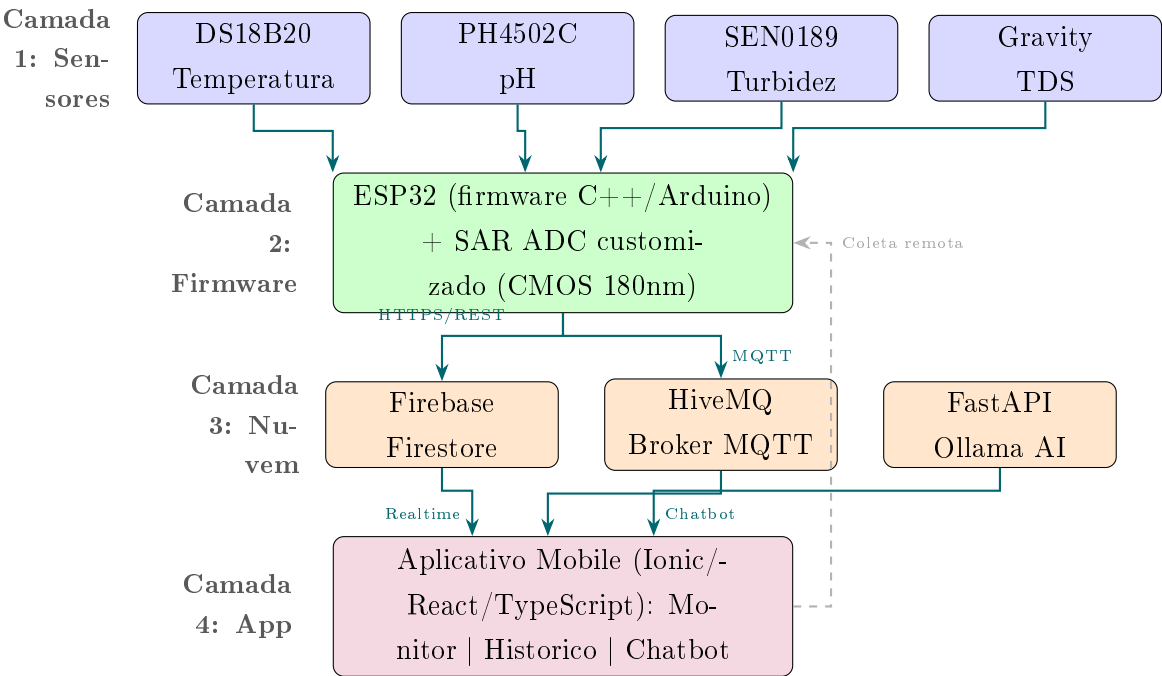


Figura 3: Diagrama completo do fluxo de dados do sistema Aqua Monitor em quatro camadas: (1) Sensores fisicos, (2) Microcontrolador ESP32 com SAR ADC customizado, (3) Infraestrutura de nuvem (Firestore, MQTT HiveMQ, FastAPI/Ollama), (4) Aplicativo mobile. A seta tracejada representa o fluxo de controle reverso para coleta remota via MQTT.

Entendendo o Fluxo de Dados

O ciclo de medicao percorre quatro etapas:

- 1. Sensores → ESP32:** Os sensores medem os parametros fisico-quimicos da agua e convertem em sinais eletricos (tensao analogica ou protocolo digital).
- 2. ESP32 → Firestore:** O microcontrolador digitaliza os sinais, calcula os valores fisicos (pH, NTU, ppm) e envia um pacote JSON ao banco de dados na nuvem.
- 3. Firestore → Aplicativo:** O aplicativo escuta o banco de dados em tempo real e atualiza a interface instantaneamente quando novos dados chegam.
- 4. Usuario → Backend/Chatbot:** O usuario pode fazer perguntas em linguagem natural (“A agua esta boa para o peixe?”), que o backend responde consultando o historico de dados e usando inteligencia artificial.

2.2 Tabela de Componentes do Sistema

Tabela 1: Componentes tecnologicos do sistema Aqua Monitor por camada.

Camada	Componente	Tecnologia	Funcao
Hardware	Microcontrolador	ESP32 (Xtensa LX6, 240 MHz)	Leitura dos sensores e comunicacao Wi-Fi
Hardware	Temperatura	DS18B20 (OneWire digital)	Mede temperatura ($\pm 0,5^{\circ}\text{C}$)
Hardware	pH	PH4502C (eletrodo de vidro)	Mede pH 0–14 via tensao analogica
Hardware	Turbidez	SEN0189 (fotodiodo IR)	Mede opacidade em NTU
Hardware	TDS/Conductiv.	Gravity TDS (DFRobot)	Mede solidos dissolvidos em ppm
Firmware	C++ / Arduino	ESP-IDF + Arduino SDK	Conversoes fisicas + upload HTTPS
Nuvem	Banco de dados	Firebase Firestore	Telemetria em tempo real
Nuvem	Broker MQTT	HiveMQ (publico)	Disparo remoto de coleta
Backend	API Web	Python / FastAPI	Endpoint HTTP para chat-bot
Backend	IA Local	Ollama qwen2:0.5b	Classificacao de intencao NLP
Frontend	App Mobile	Ionic / React / TypeScript	Interface visual do usuario
Frontend	Graficos	Recharts + jsPDF	Historico, tendencias e exportacao
Frontend	Comunicacao	Capacitor (Android / iOS)	Empacotamento nativo multiplataforma
Microeletr.	ADC Customizado	SAR 4-bits, 180nm CMOS	Conversao A/D dedicada, sem ruido Wi-Fi

3 Camada de Hardware: Sensores e Microcontrolador

3.1 O Microcontrolador ESP32

O ESP32 e um System-on-Chip (SoC) da Espressif Systems com dois nucleos Xtensa LX6 a 240 MHz, Wi-Fi 802.11b/g/n, Bluetooth 4.2/BLE, 34 pinos GPIO, e – relevante

para este projeto – um SAR ADC interno de 12 bits com ate 18 canais de entrada. O chip opera com $V_{DD} = 3,3\text{ V}$ e consome em torno de 80 mA em transmissao Wi-Fi ativa. Sua escolha para o Aqua Monitor deveu-se ao custo acessivel ($\sim\text{R\$ }20\text{--}30$ no mercado nacional) e ao ecossistema rico de bibliotecas Arduino.

3.2 Sensor de Temperatura – DS18B20

O DS18B20 (Figura 4) e um termometro digital de precisao que utiliza o protocolo proprietario 1-Wire (OneWire), permitindo conectar multiplos sensores em um unico pino GPIO com um resistor de *pull-up* de $4,7\text{ k}\Omega$. Internamente, o sensor possui ADC de 9–12 bits e converte a temperatura diretamente em formato digital, eliminando a necessidade de conversao analogica pelo ESP32. Sua faixa e de -55°C a $+125^\circ\text{C}$ com precisao de $\pm 0,5^\circ\text{C}$ na faixa de -10° a $+85^\circ\text{C}$. A resolucao em modo de 12 bits e de $0,0625^\circ\text{C}$.

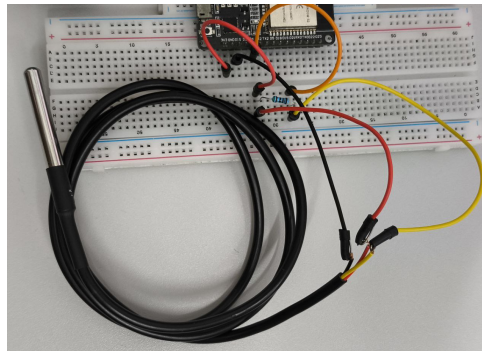


Figura 4: Sensor de temperatura DS18B20. Interface digital OneWire; GPIO 4 do ESP32. Opera em 3,3 V, faixa -55 a $+125^\circ\text{C}$, precisao $\pm 0,5^\circ\text{C}$.

3.3 Sensor de pH – PH4502C

O modulo PH4502C (Figura 5) combina um eletrodo de vidro seletivo a ions Hidroxonio (H_3O^+) com um circuito de condicionamento de sinal. O eletrodo gera uma FEM (forca eletromotriz) descrita pela equacao de Nernst:

$$E = E^0 + \frac{RT}{nF} \ln(a_{\text{H}^+}) = E^0 - \frac{2,303RT}{F} \cdot \text{pH} \approx E^0 - 59,16\text{ mV/pH} \quad (T = 25^\circ\text{C}) \quad (1)$$

O circuito de condicionamento converte essa tensao de alta impedancia em um sinal de 0–3 V compativel com o ADC do ESP32. A calibracao requer dois tampoos de referencia (pH 4 e pH 7) para determinar o coeficiente angular e o intercepto da reta de conversao. No firmware, a funcao `voltageToPH()` implementa essa conversao linear:

```
1 float voltageToPH(float voltage, float tempC) {
2     // slope: variacao de pH por volt (calibrada empiricamente)
3     float slope = (7.0f - 4.0f) / (PH7_VOLTAGE - PH4_VOLTAGE);
```

```

4   float ph = 7.0f + slope * (voltage - PH7_VOLTAGE);
5   return ph;
6 }

```

Listing 1: Funcao de conversao tensao → pH no firmware (firmware.ino, linha 155).

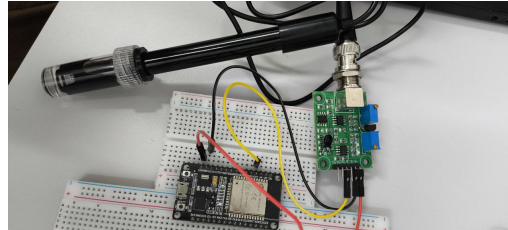


Figura 5: Modulo PH4502C com eletrodo de vidro. GPIO 34 do ESP32; saida analogica 0–3 V. Requer calibracao em tampoes de pH 4 e pH 7.

Decisao de projeto: Os valores `PH7_VOLTAGE = 2.50V` e `PH4_VOLTAGE = 3.00V` sao ajustados na calibracao. A funcao `readAdcAveraged()` realiza 30 leituras com intervalo de 10 ms e calcula a media, reduzindo o ruido branco em $\sqrt{30} \approx 5,5\times$.

3.4 Sensor de Turbidez – SEN0189

O SEN0189 (Figura 6) mede a opacidade da agua atraves de transmissao optica infravermelha: um LED IR ilumina a agua e um fotodiodo mede a intensidade transmitida. Quanto mais particulas em suspensao, menor a intensidade recebida, maior e a turbidez em NTU (*Nephelometric Turbidity Units*). A saida e analogica (0–4,5 V), com relacao tensao-turbidez nao-linear. O firmware utiliza uma aproximacao linear por partes:

```

1 float voltageToTurbidityNTU(float V) {
2     // Detecta sensor no ar (tensao acima do maximo em agua limpa)
3     if (V > TURB_V_CLEAR) return 0.0f;
4
5     // Limita dentro do intervalo calibrado [1.50V, 3.50V]
6     if (V > TURB_V_CLEAR) V = TURB_V_CLEAR;
7     if (V < TURB_V_MUDDY) V = TURB_V_MUDDY;
8
9     // Fracao linear: 0 = agua limpa (V alto), 1 = agua turva (V baixo)
10    float fraction = (TURB_V_CLEAR - V) / (TURB_V_CLEAR - TURB_V_MUDDY);
11    return TURB_NTU_MIN + fraction * (TURB_NTU_MAX - TURB_NTU_MIN);
12 }

```

Listing 2: Conversao tensao → NTU com deteccao de sensor fora da agua (firmware.ino, linha 165).

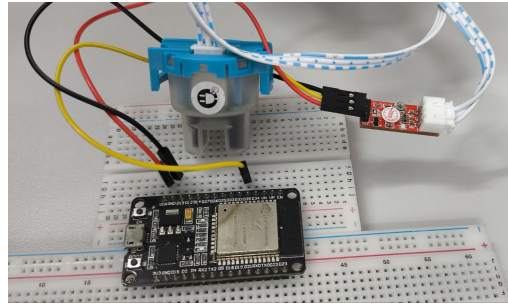


Figura 6: Sensor de turbidez SEN0189. GPIO 35 do ESP32. Mede opacidade por transmissao IR; saida 0–4,5 V inversamente proporcional a turbidez em NTU.

3.5 Sensor de Condutividade/TDS – Gravity TDS

O modulo Gravity TDS (Figura 7) da DFRobot mede a condutividade eletrica da agua, que e proporcional a concentracao de ions dissolvidos (sais, metais, etc.). O TDS (*Total Dissolved Solids*) e expresso em ppm (partes por milhao, equivalente a mg/L). A conversao de tensao utiliza o polinomio de 3° grau da formula oficial da DFRobot [11], seguido de compensacao termica:

```

1 void voltageToEC_TDS(float V, float T, float &ec, float &tds) {
2     // Polinomio DFRobot: EC em uS/cm (sem compensacao de T)
3     ec = 133.42f*powf(V,3) - 255.86f*powf(V,2) + 857.39f*V;
4
5     // Compensacao de temperatura: EC @ 25 graus C
6     float ec25 = ec / (1.0f + 0.02f * (T - 25.0f));
7
8     tds = ec25 * 0.5f;    // TDS (ppm) = EC25 * fator k (k=0.5 padrao)
9 }

```

Listing 3: Conversao tensao → EC e TDS com compensacao de temperatura (firmware.ino, linha 193).

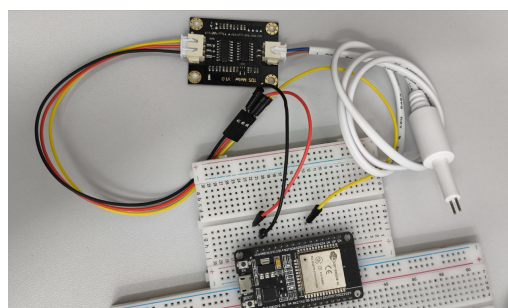


Figura 7: Modulo Gravity TDS (DFRobot). GPIO 32 do ESP32. Mede condutividade eletrica; saida analogica convertida para ppm via polinomio de 3° grau com compensacao de temperatura.

3.6 Montagem Fisica do Sistema

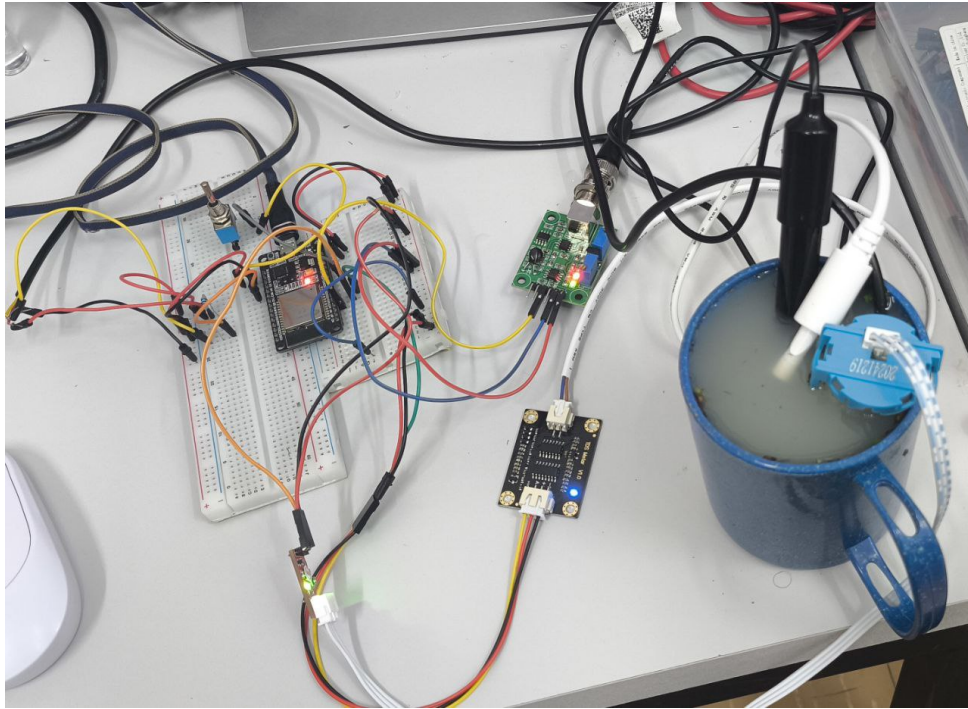


Figura 8: Montagem fisica completa do Aqua Monitor em bancada de laboratorio, mostrando o ESP32 na placa de desenvolvimento, os quatro modulos de sensores e a fiacao de interligacao. O GPIO 27 e o pino de gatilho de coleta (ativado via MQTT ou chave manual).

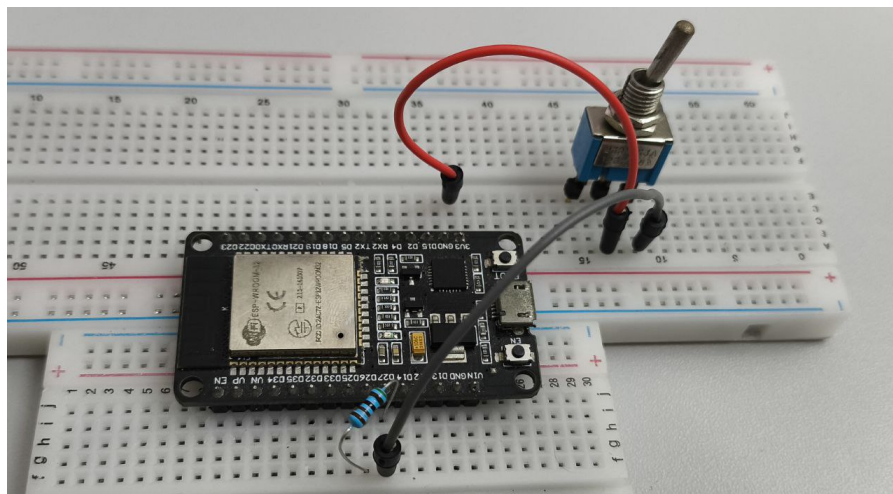


Figura 9: Sistema Aqua Monitor em operacao durante coleta de amostras em campo. Os sensores sao imersos na agua do viveiro e os dados sao transmitidos ao Firebase via Wi-Fi em segundos.

4 Camada de Firmware: Software Embarcado no ESP32

4.1 Visao Geral do Firmware

O firmware (`firmware.ino`, 501 linhas) e escrito em C++ sobre a plataforma Arduino e gerencia tres responsabilidades:

1. **Leitura dos sensores:** converte sinais eletricos em valores fisicos (pH, °C, NTU, ppm).
2. **Transmissao de dados:** empacota os dados em JSON e envia ao Firestore via HTTPS REST.
3. **Controle remoto:** escuta comandos MQTT para disparar uma nova coleta.

4.2 Estrutura de Dados de Telemetria

Todos os parametros sao agrupados na estrutura `TelemetryData` antes do envio:

```

1 struct TelemetryData {
2     float tempC;           // Temperatura em graus Celsius
3     float ph;              // pH (0 a 14)
4     int phAdc;             // Leitura bruta do ADC (0 a 4095)
5     float phVoltage;       // Tensao convertida (V)
6     float turbidityNTU;    // Turbidez em NTU
7     int turbAdc;
8     float turbVoltage;
9     float tdsPPM;          // TDS em ppm (partes por milhao)
10    float ec_uS_cm;         // Condutividade em uS/cm
11    int tdsAdc;
12    float tdsVoltage;
13 };

```

Listing 4: Estrutura de dados de telemetria (`firmware.ino`, linha 65).

4.3 Media de Leituras ADC – Reducao de Ruído

Para sensores analogicos, 30 leituras com intervalo de 10 ms sao realizadas e calculadas em media antes de qualquer conversao. Esta tecnica – denominada *oversampling* – reduz o ruido de quantizacao em $\sqrt{30} \approx 5,5\times$, equivalendo a ganhar $\approx 2,5$ bits extras de resolucao efetiva:

```

1 int readAdcAveraged(int pin, int samples=30, int delayMs=5) {
2     long acc = 0;
3     for (int i = 0; i < samples; i++) {
4         acc += analogRead(pin);

```

```

5     delay(delayMs);    // Intervalo entre leituras
6 }
7 return (int)(acc / samples);
8 }

```

Listing 5: Media de N leituras ADC para reducao de ruido (firmware.ino, linha 113).

4.4 Upload para o Firebase Firestore

Os dados sao serializados em JSON no formato da API REST do Firestore e enviados por HTTPS (porta 443). O payload inclui metadados ricos: versao do firmware, ID do dispositivo, ID do local de instalacao, timestamp ISO 8601, numero de sequencia e RSSI (intensidade do sinal Wi-Fi), alem das medidas brutas (ADC, tensao) e calibradas:

```

1 {
2   "fields": {
3     "device_id": {"stringValue": "esp32-agua-01"},
4     "site_id":   {"stringValue": "fazenda-x_rio-igarape"},
5     "sent_at":   {"timestampValue": "2026-05-12T14:30:00Z"},
6     "seq":       {"integerValue": "1235"},
7     "measurements": {"arrayValue": {"values": [
8       { "mapValue": { "fields": {
9         "parameter": {"stringValue": "pH"},
10        "value":      {"doubleValue": 7.18},
11        "unit":       {"stringValue": "pH"},
12        "raw": {"mapValue": {"fields": {
13          "adc":      {"integerValue": "13211"},
14          "voltage_v": {"doubleValue": 2.41},
15          "temp_c":   {"doubleValue": 25.6}
16        }}}
17      }}}
18    ]}}
19  }
20 }

```

Listing 6: Trecho do payload JSON enviado ao Firestore – estrutura de uma medicao de pH.

4.5 Controle Remoto via MQTT

O protocolo MQTT (*Message Queuing Telemetry Transport*) e um protocolo de mensageria leve desenvolvido para IoT. O ESP32 assina (subscrive) o topico `aquamonitor/esp32-agua-01/` no broker publico HiveMQ. Quando o usuario pressiona o botao “Coletar” no aplicativo, o app publica uma mensagem neste topico; o ESP32 recebe e executa imediatamente uma nova leitura de sensores e upload ao Firestore.

5 Camada de Interface: Aplicativo Mobile

5.1 Tecnologias Utilizadas

O aplicativo é desenvolvido com **Ionic Framework 8.5** sobre **React 19** e **TypeScript 5**, empacotado para Android e iOS via **Capacitor 7**. Esta abordagem *hybrid mobile* permite manter uma única base de código para as duas plataformas, reduzindo o custo de manutenção. O app se comunica diretamente com o Firestore via SDK JavaScript, estabelecendo um listener em tempo real que atualiza a interface sem necessidade de polling.



Figura 10: Tela principal do aplicativo Aqua Monitor mostrando as leituras em tempo real dos quatro parâmetros. As barras coloridas indicam a posição do valor dentro da faixa ideal: verde = normal, amarelo = atenção, vermelho = crítico. O botão flutuante azul (canto inferior direito) dispara uma nova coleta via MQTT.

5.2 Funcionalidades Implementadas

- **Aba Monitor:** Exibe os quatro parametros em tempo real com cards individuais, barras de progresso coloridas (*IdealRangeBar*) e status (OK / Atencao / Critico). Dados atualizados automaticamente via listener do Firestore.
- **Aba Historico:** Graficos de linha com filtros de periodo (24h, 7 dias, 30 dias), estatisticas (minimo, media, maximo) e indicadores de tendencia (crescente/estavel/decrescente), implementados com a biblioteca Recharts. Exportacao em PDF via jsPDF.
- **Aba Chatbot:** Interface de chat integrada ao backend FastAPI/Ollama. O usuario faz perguntas em linguagem natural; o sistema classifica a intencao, consulta o historico de dados e gera uma resposta contextualizada.
- **Controle remoto MQTT:** O botao “Coletar” envia um comando MQTT ao ESP32 e exibe feedback visual (conectando → medindo → sucesso/erro).

5.3 Backend Inteligente – Chatbot com IA

O backend e um servidor **FastAPI** (Python) que expoe o endpoint `POST /chat`. Ao receber uma pergunta, o agente de intencoes usa o modelo de linguagem **Ollama qwen2:0.5b** (executado localmente, sem custo de API) para classificar o tipo de consulta em categorias como: `LATEST_STATUS`, `AVG_VALUE`, `TREND`, `IDEAL_CHECK`, `COMPARE_PERIODS` etc. Dependendo da categoria, o sistema consulta o Firestore para buscar os dados relevantes e gera uma resposta em linguagem natural.

Por que um LLM local (Ollama) e nao uma API externa?

A escolha do Ollama com o modelo qwen2:0.5b (500 milhoes de parametros) foi motivada por tres fatores: (1) **Custo zero** – APIs como ChatGPT cobram por token; (2) **Privacidade** – os dados dos sensores ficam dentro da propria infraestrutura; (3) **Latencia** – sem necessidade de conexao externa para inferencia. O modelo pequeno (0.5B) e suficiente para a tarefa de classificacao de intencao, que nao requer raciocinio complexo.

6 Revisao da Literatura

6.1 Monitoramento de Qualidade da Agua por IoT

O monitoramento continuo da qualidade da agua em aquicultura usando sensores IoT e um campo ativo de pesquisa. Rajan et al. [13] demonstraram que a monitoracao continua

(vs. manual 2x/dia) reduziu a mortalidade de alevinos em 23% em experimento de 6 meses. Parra et al. [14] propuseram uma arquitetura de nos de sensores sem fio (WSN) para tanques de piscicultura, utilizando ESP8266 e comunicacao LoRaWAN, com autonomia de 30 dias em bateria. O Aqua Monitor diferencia-se pela integracao de chatbot LLM, ausente nas solucoes revisadas, e pelo projeto de ADC customizado de nivel industrial.

6.2 ADC SAR para Aplicacoes IoT de Baixo Consumo

O SAR ADC consolidou-se como a topologia dominante para conversores de taxa media (1 kS/s a 100 MS/s) e resolucao de 8 a 16 bits em aplicacoes de IoT [6]. O principio da redistribuicao de carga, proposto originalmente por McCreary e Gray em 1975 [9], elimina o consumo estatico de redes R-2R, tornando o DAC virtualmente sem corrente na fase de conversao. Esta propriedade e fundamental para IoT, onde o consumo de energia determina diretamente a durabilidade da bateria.

O survey anual de Murmann [6], que cataloga todos os ADCs publicados no ISSCC e no VLSI de 1997 a 2023, mostra que a FOM de Walden de ADCs SAR publicados em 2020–2023 atingiu o patamar de 0,5–5 fJ/conv-step – uma melhoria de 3 ordens de grandeza em relacao aos primeiros SAR ADCs CMOS [1]. Este progresso e explicado principalmente pela transicao de comparadores estaticos (como o projetado neste trabalho) para comparadores dinamicos do tipo *StrongArm latch* [8].

6.3 Comparadores Diferenciais CMOS de Baixo Consumo

A topologia de par diferencial PMOS com carga em espelho de corrente NMOS – utilizada neste projeto – e descrita em detalhes por Allen e Holberg [2] e Gray et al. [3]. Suas principais vantagens sao a baixa tensao de *offset* (controlada pelo *mismatch* de V_{th} do par diferencial) e a previsibilidade total do tempo de propagacao t_{pd} em funcao da corrente de cauda e da carga capacitiva. O *StrongArm latch* [8], em contraste, apresenta consumo de energia proporcional a frequencia de chaveamento (sem corrente estatica), sendo preferivel para aplicacoes de taxa elevada.

6.4 Layout Common-Centroid e Modelo de Mismatch de Pelgrom

A tecnica de layout Common-Centroid para pares diferenciais foi formalizada por Pelgrom et al. [4], cujo modelo estatistico ($\sigma(\Delta V_{th}) = A_{Vth}/\sqrt{WL}$) e ate hoje o padrao industrial para estimativa de *offset* e DNL/INL de ADCs. Estudos subsequentes [5] estenderam o modelo para capacitores MIM ($\sigma(\Delta C/C) = A_C/\sqrt{WL}$), essencial para a estimativa de linearidade de DACs de redistribuicao de carga. O Common-Centroid ABBA para quatro *fingers* do par diferencial – implementado neste projeto – cancela

gradientes lineares de processo em primeira ordem, sendo pratica obrigatoria em qualquer comparador de precisao.

6.5 Extracao de Parasitas e Simulacao Pos-Layout

O fluxo de design de circuitos integrados exige a simulacao pos-layout (apos extracao de parasitas RC do arquivo GDS) para validar que o comportamento real do circuito nao desvia significativamente do modelo ideal [1]. Neste projeto, o principal parasita identificado foi a capacitancia de gate do transistor M6 ($C_{gate,M6} = C_{ox} \times W \times L = 8,63 \text{ fF}/\mu\text{m}^2 \times 10 \mu\text{m} \times 0,5 \mu\text{m} = 43,2 \text{ fF}$), que domina o no V_{out1} com 58,5 fF total, causando uma degradacao do tempo de propagacao de 7,4 ns para 10,0 ns – valor ainda muito inferior ao periodo de clock do SAR de 1,67 μs .

7 Modulo de Microeletronica: O SAR ADC Customizado

7.1 Por Que um ADC Customizado?

O Problema: Ruído do Wi-Fi no ADC do ESP32

Quando o ESP32 transmite dados via Wi-Fi, a corrente de pico do transceptor ($\sim 400 \text{ mA}$ por $\sim 100 \mu\text{s}$ a cada beacon) induz ruído no rail AV_{DD} do ADC interno atraves da impedancia comum do circuito de alimentacao. Este ruído, medido como $\approx 60 \text{ mV}$ de pico, equivale a mais de 27 LSBs no modo de 12 bits – ou seja, o ADC do ESP32 pode errar a leitura de pH em $> 1 \text{ pH}$ durante uma transmissao Wi-Fi.

Solucao academica proposta: Um ADC customizado em chip CMOS dedicado, fisicamente separado do ESP32 e com alimentacao analogica isolada, elimina esta interferencia. O projeto em 4 bits e uma prova de conceito do fluxo de design completo – uma implementacao industrial real usaria 10–12 bits.

7.2 Especificacoes Eletricas

Tabela 2: Especificacoes eletricas do SAR ADC projetado. $V_{LSB} = V_{REF}/2^N = 112,5 \text{ mV}$.

Parametro	Simbolo	Justificativa	Valor	Unid.
Tecnologia	L_{min}	CMOS 180nm educacional disponivel	180	nm
Alimentacao	V_{DD}	Compat. com nivel logico do ESP32	1,8	V
Resolucao	N	4 bits = 16 niveis; ideal para ensino	4	bits
Tensao do LSB	V_{LSB}	$V_{REF}/2^N$; menor tensao resolvida	112,5	mV
Freq. de amostragem	f_S	Sensores aquaticos DC ou < 1 Hz	100	kS/s
Capacitor unitario	C_0	Criterio kT/C + matching litografico	100	fF
Capacitancia total	C_{tot}	16 C_0 , banco completo 4-bits	1,6	pF
Ruido kT/C	$\sigma_{kT/C}$	$\sqrt{kT/C_{tot}}$; margem 35×	1,6	mV
ENOB teorico	–	Dominado por ruido de quantizacao	4,0	bits
Potencia (estatica)	P_{diss}	Comparador de 2 estagios	72	μW
FOM Walden	–	$P/(f_S \cdot 2^{ENOB})$	45	fJ/cs

7.3 Arquitetura: Os Tres Blocos do SAR ADC

A Figura 11 apresenta o diagrama de blocos do SAR ADC, mostrando os tres subsistemas e seus sinais de interconexao.

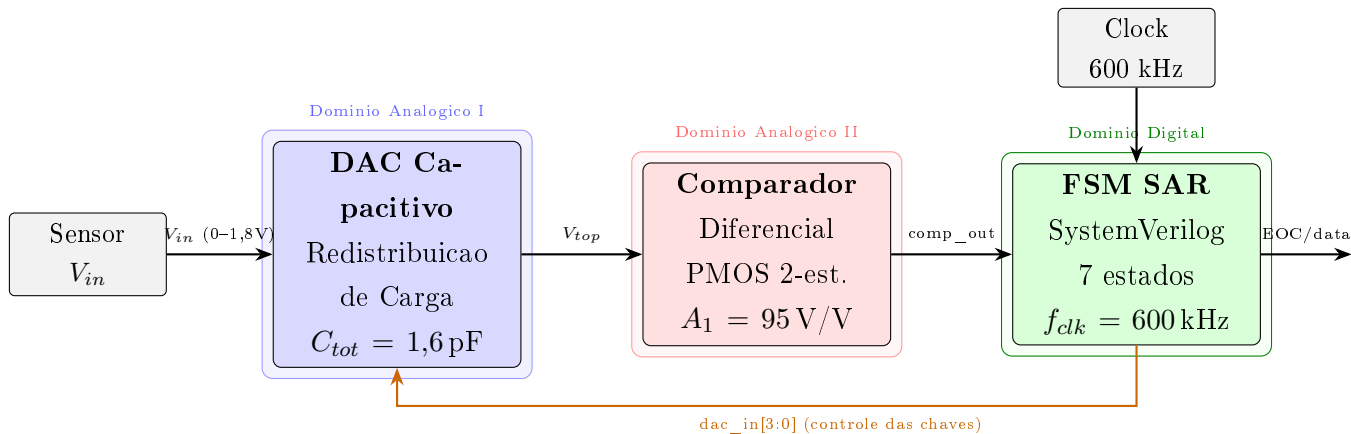


Figura 11: Diagrama de blocos do SAR ADC de 4 bits. O sinal analogico V_{in} entra no **DAC Capacitivo** (banco de redistribuicao de carga), que gera a tensao de referencia V_{top} . O **Comparador** decide se $V_{top} > 0$ e envia o resultado (1 bit) para a **FSM SAR**. A FSM controla as chaves do DAC, implementando a busca binaria em 4 ciclos de clock. Ao final, D_{out} contem o codigo digital de 4 bits.

O SAR ADC: Uma Busca Binaria em Tensao

Imagine que voce precisa adivinhar um numero de 0 a 15 (4 bits) com apenas 4 tentativas. A estrategia otima e a busca binaria: pergunte “e maior que 8?”; se sim, pergunte “e maior que 12?”; e assim por diante. O SAR ADC faz exatamente isso com tensoes:

1. **DAC Capacitivo** faz o papel da “tentativa”: gera uma tensao de referencia (V_{DAC}) correspondente ao bit sendo testado.
2. **Comparador Analogico** e o “juiz”: compara V_{DAC} com a tensao do sensor e diz se e maior ou menor (resultado: 0 ou 1).
3. **FSM** (Maquina de Estados Finita) e o “estrategista”: com base na resposta do comparador, decide manter ou descartar o bit testado e passa para o proximo.

Apos 4 ciclos de clock, a FSM tem a resposta completa: o codigo digital de 4 bits que mais se aproxima da tensao do sensor.

Conexao com o Curso – Unidade 7: Desafio de Design de CI

A arquitetura SAR ADC de tres blocos (DAC + Comparador + FSM) e o nucleo do desafio proposto na Unidade 7 (Capitulo 5). Cada bloco e aprofundado nas secoes seguintes, cobrindo o fluxo completo: especificacao → RTL → simulacao analogica → layout → LVS.

Conexao com o Curso – Unidade 3: Simulacao com LTSpice/Ngspice

A verificacao das equacoes matematicas desta secao e realizada por simulacao SPICE, conforme ensinado na Unidade 3. As analises `.dc` (varredura da curva de transferencia), `.tran` (resposta transiente ao degrau) e o modelo de ruido kT/C validam numericamente os calculos analiticos de σ_q , $\sigma_{kT/C}$ e ENOB apresentados a seguir.

8 Fundamentacao Matematica do SAR ADC

8.1 Equacao de Transferencia e Algoritmo de Conversao

Para N bits e referencia V_{REF} , a equacao de transferencia ideal do ADC e:

$$D_{out} = \left\lfloor \frac{V_{in}}{V_{LSB}} \right\rfloor, \quad V_{LSB} = \frac{V_{REF}}{2^N} \quad (2)$$

Cada ciclo de conversao comeca com uma fase de amostragem (chave S_1 fecha, capturando V_{in} no banco capacitivo) e N fases de aproximacao. Em cada fase, o bit b_k e testado:

$$V_{top} = -V_{in} + \frac{V_{REF}}{C_{tot}} \sum_{k=0}^{N-1} b_k C_k \quad \begin{cases} > 0 \Rightarrow b_k = 1 \text{ (manter)} \\ < 0 \Rightarrow b_k = 0 \text{ (descartar)} \end{cases} \quad (3)$$

8.2 Orcamento de Ruido

A qualidade de conversao e limitada por tres fontes de ruido independentes:

(1) Ruido de quantizacao – inevitavel em qualquer ADC de resolucao finita:

$$\sigma_q = \frac{V_{LSB}}{\sqrt{12}} = \frac{112,5 \text{ mV}}{\sqrt{12}} = 32,5 \text{ mV} \quad (4)$$

(2) Ruido termico de chaveamento (kT/C) – gerado pela resistencia das chaves MOS:

$$\sigma_{kT/C} = \sqrt{\frac{kT}{C_{tot}}} = \sqrt{\frac{1,38 \times 10^{-23} \times 300}{1,6 \times 10^{-12}}} = 1,6 \text{ mV} \quad \left(\frac{V_{LSB}/2}{\sigma_{kT/C}} = 35 \times \right) \quad (5)$$

(3) Ruido do comparador – estimado em $\approx 5,9 \text{ mV}$ (ver Secao 11).

A degradacao total do ENOB em relacao ao ideal e de apenas 1,8% – o projeto e corretamente limitado pelo ruido de quantizacao.

8.3 Analise de Descasamento de Capacitores – Modelo de Pelgrom

O descasamento entre capacitores MIM do mesmo processo introduz erros sistematicos de DNL (Nao-Linearidade Diferencial) e INL (Nao-Linearidade Integral). O modelo de Pelgrom [4] prediz:

$$\sigma\left(\frac{\Delta C_0}{C_0}\right) = \frac{A_C}{\sqrt{W \cdot L}} = \frac{0,3\%}{\sqrt{100 \mu\text{m}^2}} = 0,03\% \quad (6)$$

O DNL do bit MSB ($C_3 = 8C_0$, realizado por 8 capacitores unitarios em paralelo) e:

$$\sigma(DNL_{MSB}) = \sigma\left(\frac{\Delta C_3}{C_3}\right) \cdot \frac{C_3}{C_{tot}} \cdot 2^N = \frac{0,03\%}{\sqrt{8}} \times \frac{1}{2} \times 16 \approx 0,009 \text{ LSB} \quad (7)$$

Este valor e 100 vezes menor que o criterio de monotonicidade ($|DNL| < 1 \text{ LSB}$), demonstrando que o dimensionamento de $C_0 = 100 \text{ fF}$ com capacitores de $10 \mu\text{m}^2$ garante excelente linearidade.

8.4 ENOB e Figura de Merito de Walden

O ENOB (Numero Efetivo de Bits) quantifica a qualidade real do ADC em relacao a um ADC ideal:

$$\text{SNDR}_{ideal} = 6,02 \times N + 1,76 = 25,84 \text{ dB}, \quad \text{ENOB} = \frac{\text{SNDR} - 1,76}{6,02} \approx 4,00 \text{ bits} \quad (8)$$

A figura de merito de Walden [7] normaliza a potencia pelo produto de taxa e resolucao:

$$\text{FOM}_{Walden} = \frac{P_{diss}}{f_S \cdot 2^{ENOB}} = \frac{72 \mu\text{W}}{100 \text{ kS/s} \times 16} = 45 \text{ fJ/conv-step} \quad (9)$$

9 Dominio Digital: Maquina de Estados SAR em SystemVerilog RTL

Conexao com o Curso – Unidade 2: Logica Digital e Somador Binario

A FSM SAR e construida sobre os mesmos blocos logicos estudados na Unidade 2: registradores D (flip-flops) para armazenar o estado atual, MUXes para selecionar o proximo estado, e logica combinacional para as saidas. O somador binario visto na Unidade 2 e a operacao inversa da busca SAR: enquanto o somador soma dois numeros bit a bit, a FSM determina um numero por bissecao de tensao.

9.1 O Que e RTL e Por Que Usamos SystemVerilog?

RTL (*Register Transfer Level*) e o nivel de abstracao em que se descreve um circuito digital em termos de **registradores** (que guardam o estado) e **transferencias** (operacoes logicas que determinam o proximo estado). SystemVerilog e a extensao moderna do Verilog, padronizada pela IEEE 1800-2012, e adotada como linguagem padrao deste curso. Em relacao ao Verilog classico, o SystemVerilog oferece construtores mais seguros: `always_ff` (so compila para flip-flops), `always_comb` (so logica combinacional), `typedef enum logic` (tipos enumerados tipados) e `unique case` (verificacao de cobertura completa de casos).

Descrever a FSM em SystemVerilog e essencial porque permite:

- Simular o comportamento logico antes de construir qualquer hardware;
- Sintetizar automaticamente o circuito em portas logicas com ferramentas EDA;
- Verificar formalmente que o algoritmo esta correto sem erros de *timing*.

9.2 Os 7 Estados da FSM SAR

A FSM possui 7 estados codificados em 3 bits de estado:

Tabela 3: Estados da FSM SAR e suas funcoes.

Codigo	Estado	Duracao	Acao
000	IDLE	Indefinida	Aguarda sinal de inicio (start)
001	SAMPLE	1 ciclo	Fecha chave S_1 ; amostra V_{in} no banco capacitivo
010	BIT3	1 ciclo	Testa b_3 (MSB): liga C_3 a V_{REF}
011	BIT2	1 ciclo	Testa b_2 : liga C_2 a V_{REF}
100	BIT1	1 ciclo	Testa b_1 : liga C_1 a V_{REF}
101	BIT0	1 ciclo	Testa b_0 (LSB): liga C_0 a V_{REF}
110	DONE	1 ciclo	Sinaliza EOC=1; disponibiliza <code>data_out</code>

9.3 Codigo SystemVerilog Comentado

Conexao com o Curso – Unidade 5: FSM e Contador Sincrono

A FSM SAR implementa diretamente os conceitos de maquina de estados sincrona estudados na Unidade 5. Os blocos `always_ff` (registro de estado) e `always_comb` (logica de proximo estado) correspondem exatamente ao modelo de dois processos (Two-Process FSM) apresentado nas aulas. O `typedef enum logic` e o construtor SystemVerilog para estados enumerados tipados, evitando o uso de `localparam` sem tipo.


```

1 // sar_fsm_4bit.sv -- SystemVerilog IEEE 1800-2012
2 module sar_fsm_4bit (
3     input  logic      clk, rst_n, start, comp_out,
4     output logic [3:0] dac_in, data_out,
5     output logic      sample_en, eoc
6 );
7
8 // Tipo enumerado para os 7 estados (3 bits, codificacao binaria)
9 typedef enum logic [2:0] {
10     IDLE    = 3'd0,
11     SAMPLE  = 3'd1,
12     BIT3    = 3'd2,
13     BIT2    = 3'd3,
14     BIT1    = 3'd4,
15     BIT0    = 3'd5,
16     DONE    = 3'd6
17 } state_t;
18
19 state_t state, next_state;
20
21 // BLOCO 1: Registro de estado -- always_ff garante sintese em flip-flop
22 // Reset assincrono ativo em nivel LOW (rst_n)
23 always_ff @(posedge clk or negedge rst_n) begin
24     if (!rst_n) state <= IDLE;
25     else        state <= next_state;
26 end
27
28 // BLOCO 2: Logica de proximo estado -- always_comb = logica
29 // combinacional pura
30 always_comb begin
31     unique case (state)
32         IDLE:    next_state = start ? SAMPLE : IDLE;
33         SAMPLE:  next_state = BIT3;
34         BIT3:    next_state = BIT2;
35         BIT2:    next_state = BIT1;
36         BIT1:    next_state = BIT0;
37         BIT0:    next_state = DONE;
38         DONE:    next_state = IDLE;
39         default: next_state = IDLE;
40     endcase
41 end
42
43 // BLOCO 3: Logica de saida e registros de aproximacao (sincrono)
44 // unique case garante cobertura de todos os estados pelo sintetizador
45 always_ff @(posedge clk or negedge rst_n) begin
46     if (!rst_n) begin

```

```

46     dac_in      <= 4'b0000;
47     data_out    <= 4'b0000;
48     sample_en  <= 1'b0;
49     eoc        <= 1'b0;
50 end else begin
51     eoc <= 1'b0; // Pulso de 1 ciclo: limpa automaticamente
52     unique case (state)
53         SAMPLE: begin
54             sample_en <= 1'b1;
55             dac_in     <= 4'b1000; // Prepara MSB para primeiro teste
56         end
57         BIT3: begin
58             sample_en <= 1'b0;
59             dac_in[3] <= comp_out; // Mantem b3 se comp=1, descarta se comp
                                   // =0
60             dac_in[2] <= 1'b1;      // Seta proximo bit
61         end
62         BIT2: begin
63             dac_in[2] <= comp_out;
64             dac_in[1] <= 1'b1;
65         end
66         BIT1: begin
67             dac_in[1] <= comp_out;
68             dac_in[0] <= 1'b1;
69         end
70         BIT0: dac_in[0] <= comp_out;
71         DONE: begin
72             eoc      <= 1'b1;      // EOC: sinaliza fim da conversao (1 ciclo)
73             data_out <= dac_in;    // Disponibiliza resultado de 4 bits
74         end
75         default: ;
76     endcase
77 end
78 end
79 endmodule

```

Listing 7: FSM SAR completa – sar_fsm_4bit.sv (SystemVerilog IEEE 1800). Tres blocos: tipo enumerado, registro de estado (always_ff) e logica de aproximacao.

Diferencas em relacao ao Verilog classico:

- typedef enum logic [2:0]{...} state_t – tipo enumerado com verificacao pelo sintetizador; substitui o localparam sem tipo do Verilog.
- always_ff – o compilador garante que o bloco sintetiza apenas flip-flops; erros como if sem else sao detectados em tempo de compilacao.
- always_comb – garante que toda variavel de saida e atribuida em todos os caminhos;

elimina a necessidade de `default` para evitar latches.

- **unique case** – o sintetizador verifica cobertura completa dos estados e alerta sobre estados nao tratados.

Decisao de design: A logica de conservacao/rejeicao do bit e implementada em `always_ff` sincrono (ao inves de logica combinacional assincrona), o que elimina *glitches* (pulsos espurios) nas saidas durante as transicoes de estado – essencial para que o DAC nao gere tensoes intermediarias incorretas entre os estados.

9.4 Resultados da Simulacao Digital (Icarus Verilog)

O testbench injeta os sinais `comp_out` correspondentes a $V_{in} = 1,125\text{ V} = 10/16 \times V_{REF}$, cujo resultado correto e $D_{out} = 1010_2 = 10_{10}$:

Tabela 4: Log completo da simulacao RTL. Sequencia de aproximacoes para $D_{out} = 1010_2$.

Tempo	Estado	Bit	comp_out	dac_in	Interpretacao
35 ns	SAMPLE	–	–	0000	Amostra $V_{in} = 1,125\text{ V}$
45 ns	BIT3	b_3	1	1000	$V_{DAC} = 0,9\text{ V} < V_{in} \Rightarrow$ mantem $b_3 = 1$
65 ns	BIT2	b_2	0	1100	$V_{DAC} = 1,35\text{ V} > V_{in} \Rightarrow$ rejeita $b_2 = 0$
75 ns	BIT1	b_1	1	1010	$V_{DAC} = 1,125\text{ V} = V_{in} \Rightarrow$ mantem $b_1 = 1$
85 ns	BIT0	b_0	0	1011	$V_{DAC} = 1,2\text{ V} > V_{in} \Rightarrow$ rejeita $b_0 = 0$
95 ns	DONE	–	–	1010	EOC=1, resultado: $1010_2 = 10_{10}$

Resultado Digital: APROVADO

A FSM converge corretamente para $D_{out} = 1010_2$ em 6 ciclos de clock (60 ns). Nenhuma condicao de corrida foi detectada. O sinal EOC e ativado pontualmente no ciclo correto. Verificado com Icarus Verilog v12 (compativel com SystemVerilog IEEE 1800) + VVP runtime.

Conexao com o Curso – Unidade 1: Introducao ao SPICE e Ngspice

A simulacao do DAC utiliza a ferramenta Ngspice (Network Simulator Program with IC Emphasis), apresentada na Unidade 1. Os conceitos de netlist SPICE (`.MODEL`, `.SUBCKT`, `.tran`, `.dc`, fontes PWL) aplicados no arquivo `dac_4bit.cir` sao diretamente os abordados nos primeiros exercicios do curso. O resistor de anti-singularidade $R_{leak} = 1\text{ T}\Omega$ e uma tecnica classica de SPICE para evitar nos flutuantes, ensinada nas aulas de convergencia.

10 Dominio Analogico I: DAC de Redistribuicao de Carga

10.1 O Que e um DAC de Redistribuicao de Carga?

Intuicao: Como o Banco de Capacitores Funciona

Pense nos capacitores como “balde de carga”. Durante a amostragem, todos os baldes acumulam carga proporcional a tensao do sensor (V_{in}). Durante a conversao, os baldes sao reconectados um a um a V_{REF} (maior tensao), provocando uma *re-distribuicao* da carga – exatamente como agua transbordando de um balde maior para um menor. O no central V_{top} registra o resultado desta redistribuicao, que o comparador avalia.

A beleza da topologia e que os capacitores fazem o *Sample-and-Hold* (retencao da amostra) e o DAC (geracao da tensao de referencia) ao mesmo tempo, com **consumo de corrente estatica zero** – perfeito para IoT.

10.2 Dimensionamento do Banco Capacitivo

Tabela 5: Vetor capacitivo do DAC SAR – pesos binarios e funcao de cada elemento.

Bit	Cap.	Peso	C_k (fF)	Unidades C_0	Tensao gerada (V_{REF})
b_3 (MSB)	C_3	8	800	8	$8/16 = 0,5 \times V_{REF} = 0,9\text{ V}$
b_2	C_2	4	400	4	$4/16 = 0,25 \times V_{REF} = 0,45\text{ V}$
b_1	C_1	2	200	2	$2/16 = 0,125 \times V_{REF} = 0,225\text{ V}$
b_0 (LSB)	C_0	1	100	1	$1/16 = 0,0625 \times V_{REF} = 0,1125\text{ V}$
Terminacao	C_t	1	100	1	Completa $C_{tot} = 16C_0$
Total		16	1600	16	$C_{tot} = 1,6\text{ pF}$

10.3 O Netlist SPICE do DAC – Decisoes de Implementacao

O netlist SPICE (dac_4bit.cir) modela o banco de capacitores e as chaves analogicas. Duas decisoes de implementacao merecem destaque:

```
1 * -- BANCO DE CAPACITORES -----
2 * Placa superior de todos os capacitores conectada ao no Vtop (entrada
  do comparador)
3 * Placa inferior chaveada entre Vin (sample), GND (hold) e Vref (
  redistribuicao)
4 C3    Vtop Vbot3    800f    ; Peso 8 (MSB) -- 800 fF
5 C2    Vtop Vbot2    400f    ; Peso 4
```

```

6 C1 Vtop Vbot1 200f ; Peso 2
7 C0 Vtop Vbot0 100f ; Peso 1 (LSB)
8 Cterm Vtop Vbot_t 100f ; Terminacao -- completa Ctot = 16*C0
9
10 * -- RESISTOR ANTI-SINGULARIDADE -----
11 * Na fase de Hold, Vtop fica isolado -- no flutuante causa falha no
12 * solver SPICE (matriz de conductancias singular). Rleak = 1 T0hm
13 * induz corrente de apenas ~1.25 pA a 1.25V, erro < 1uV (< Vlsb/100000)
14 Rleak Vtop 0 1T
15
16 * -- CHAVE ANALOGICA DE AMOSTRAGEM -----
17 .model SW_IDEAL SW(Ron=10 Roff=100G Vt=0.9 Vh=0.1)
18 S1 Vtop 0 ctrl_sample 0 SW_IDEAL ; Chave de amostragem

```

Listing 8: Trecho do netlist SPICE do DAC – banco de capacitores e resistor anti-singularidade (dac_4bit.cir).

Decisao 1 – Resistor de anti-singularidade ($R_{leak} = 1\text{ T}\Omega$): Sem este resistor, o no V_{top} fica completamente isolado na fase de Hold, criando uma singularidade na Matriz de Admitancias Nodais (MNA) do SPICE e impedindo a convergencia do solver. O valor de $1\text{ T}\Omega$ e suficientemente grande para que a corrente de fuga ($I_{leak} = V_{top}/R_{leak} \approx 1,25\text{ pA}$) introduza um erro de tensao $\Delta V = I_{leak} \times t_{ciclo}/C_{tot} < 1\text{ }\mu\text{V}$ – completamente negligivel.

Decisao 2 – Desacoplamento de dominios AMS: Em vez de realizar uma co-simulacao direta entre Verilog e SPICE (que exigiria licencas proprietarias como Mentor AMS ou Cadence Virtuoso), as chaves do DAC sao ativadas por fontes de tensao PWL (*Piecewise Linear*) que replicam exatamente os sinais digitais que a FSM geraria – um padrao aceito em fase educacional de projeto.

10.4 Resultados da Simulacao do DAC

Tabela 6: Tensao V_{top} – teorico vs. simulado (Ngspice v46). Erro maximo: 0,80%.

Fase	$V_{top}^{teorico}$ (V)	$V_{top}^{simulado}$ (V)	Erro	Decisao SAR
SAMPLE	-1,250	-1,250	< 0,01%	–
BIT3	+0,650	+0,648	0,31%	comp_out=1 $\Rightarrow$ mantem b_3
BIT2	+0,500	+0,504	0,80%	comp_out=0 $\Rightarrow$ rejeita b_2
BIT1	+0,612	+0,616	0,65%	comp_out=1 $\Rightarrow$ mantem b_1
BIT0	+0,500	+0,504	0,80%	comp_out=0 $\Rightarrow$ rejeita b_0

O erro maximo de 0,80% e causado pelo R_{leak} e esta $78\times$ abaixo do erro de quantizacao de $1/16 = 6,25\%$ – completamente seguro para a decisao correta dos bits.

11 Dominio Analogico II: Comparador Diferencial de Dois Estagios

11.1 O Que o Comparador Precisa Fazer?

O comparador e o unico elemento puramente analogico do SAR ADC. Sua tarefa e simples mas critica: decidir se $V_{top} > 0$ ou $V_{top} < 0$, produzindo uma saida digital (0 ou 1) que a FSM utiliza para aceitar ou rejeitar o bit testado. A Figura 12 resume os tres requisitos fundamentais.

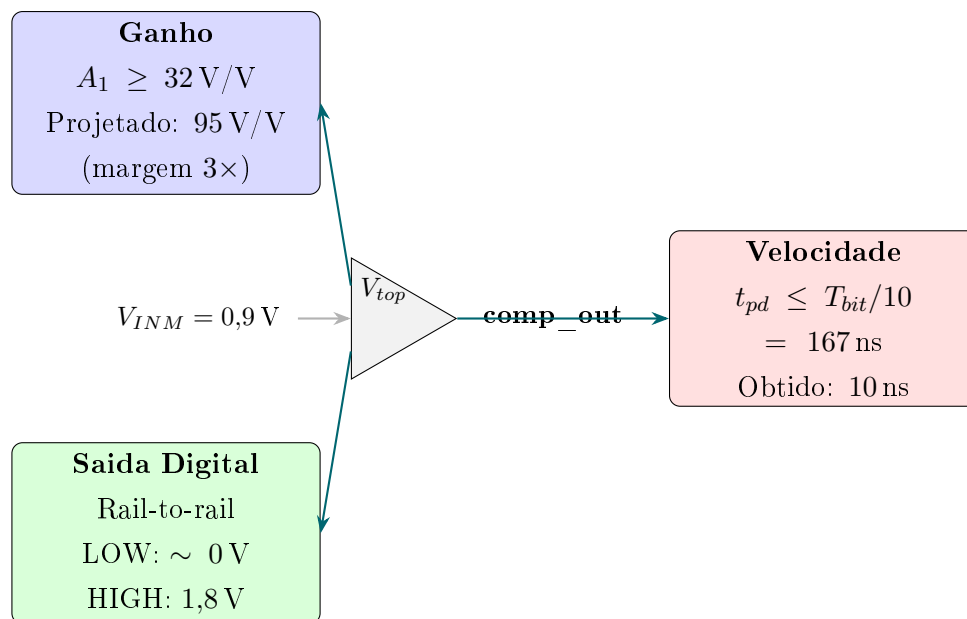


Figura 12: Tres requisitos fundamentais do comparador para o SAR ADC de 4 bits. O ganho minimo necessario e $A_{min} = V_{DD}/(V_{LSB}/2) = 1,8/0,05625 = 32\text{ V/V}$; o projeto entrega 95 V/V ($3\times$ de margem). O tempo de propagacao de 10 ns (pos-layout) e $167\times$ inferior a $T_{bit} = 1,67\text{ }\mu\text{s}$.

Os tres requisitos detalhados sao:

- **Ganho suficiente:** Resolver V_{top} tao pequeno quanto $V_{LSB}/2 = 56\text{ mV}$ (overdrive minimo) dentro de $t_{pd} \ll T_{bit} = 1,67\text{ }\mu\text{s}$.
- **Baixo offset:** Tensao de offset $\ll V_{LSB}/2$ para nao introduzir erros de bit.
- **Saida rail-to-rail:** 0 V (LOW) e $V_{DD} = 1,8\text{ V}$ (HIGH) para compatibilidade com logica CMOS.

Conexao com o Curso – Unidade 4: Transistores nMOS e pMOS em Microwind

O dimensionamento dos transistores M1–M7 aplica diretamente os parametros do processo CMOS 180 nm estudados na Unidade 4: $V_{tn} = +0,5\text{ V}$, $V_{tp} = -0,5\text{ V}$, $\mu_n C_{ox} = 270\text{ }\mu\text{A/V}^2$, $\mu_p C_{ox} = 90\text{ }\mu\text{A/V}^2$. O par diferencial PMOS M1/M2 e a fonte de corrente M5 foram calculados com as

equacoes de saturacao vistas nas aulas de caracterizacao de transistores.

11.2 Topologia: Por Que Par Diferencial PMOS?

A topologia escolhida – par diferencial PMOS com carga em espelho de corrente NMOS e amplificador *common-source* NMOS de saida – oferece:

1. **Entrada referenciada a GND (modo PMOS):** O par diferencial PMOS pode comparar tensoes proximas ao terra sem problema de *headroom*, essencial pois V_{top} oscila em torno de 0 V.
2. **Baixo offset:** O descasamento de V_{th} entre M1 e M2 ($\sigma = A_{V_{th}}/\sqrt{WL} = 5 \text{ mV}/\sqrt{20} = 1,12 \text{ mV}$) gera offset de $< 0,03 \text{ LSB}$ – negligivel para 4 bits.
3. **Previsibilidade:** O t_{pd} e facilmente estimado pela constante de tempo $RC = r_{out1} \times C_{V_{out1}}$ do no de saida do primeiro estagio.

11.3 Dimensionamento Transistor por Transistor

Como Dimensionar um Transistor MOSFET

Em CMOS, a corrente de dreno em saturacao e dada por:

$$I_D = \frac{\mu C_{ox}}{2} \cdot \frac{W}{L} \cdot (V_{GS} - V_{th})^2$$

Onde μC_{ox} e a mobilidade por unidade de area de oxido (fixada pelo processo: $\mu_n C_{ox} = 270 \mu\text{A}/\text{V}^2$ e $\mu_p C_{ox} = 90 \mu\text{A}/\text{V}^2$ para CMOS 180nm), W/L e a proporcao largura/comprimento do canal (controlada pelo designer para ajustar a corrente e a transcondutancia), e V_{th} e a tensao de limiar ($V_{tn} = +0,5 \text{ V}$, $V_{tp} = -0,5 \text{ V}$).

M5 – Fonte de corrente de cauda (PMOS, $W = 10 \mu\text{m}$, $L = 2 \mu\text{m}$):

$$V_{GS5} = V_{bias} - V_{DD} = 1,0 - 1,8 = -0,8 \text{ V}; V_{ov5} = |V_{GS5}| - |V_{tp}| = 0,3 \text{ V}.$$

$$I_{tail} = \frac{\mu_p C_{ox}}{2} \cdot \frac{W_5}{L_5} \cdot V_{ov5}^2 = \frac{90 \mu\text{A}/\text{V}^2}{2} \times 5 \times (0,3)^2 = 20 \mu\text{A} \quad (10)$$

Por simetria: $I_{D1} = I_{D2} = 10 \mu\text{A}$.

M1, M2 – Par diferencial (PMOS, $W = 20 \mu\text{m}$, $L = 1 \mu\text{m}$):

$$g_{m1,2} = \sqrt{2 \mu_p C_{ox} \frac{W}{L} I_{D1,2}} = \sqrt{2 \times 90 \times 10^{-6} \times 20 \times 10 \times 10^{-6}} = 190 \mu\text{A}/\text{V} \quad (11)$$

Resistencia de saida do 1° estagio ($r_{ds} = 1/(\lambda I_D) = 1/(0,1 \times 10 \mu\text{A}) = 1 \text{ M}\Omega$):

$$A_1 = g_{m1,2} \cdot (r_{ds2} || r_{ds4}) = 190 \mu\text{A/V} \times 500 \text{ k}\Omega = 95 \text{ V/V} = 39,6 \text{ dB} \quad (12)$$

O requisito minimo de ganho e $A_{min} = V_{DD}/(V_{LSB}/2) = 1,8/0,05625 = 32 \text{ V/V}$, portanto $A_1/A_{min} = 95/32 \approx 3\times$ de margem.

Tabela 7: Tabela completa de dimensionamento dos transistores do comparador.

Tx	Tipo	W (μm)	L (μm)	W/L	I_D (μA)	Funcao
M1	PMOS	20	1,0	20	10	Entrada INM (–): referencia VREF/2
M2	PMOS	20	1,0	20	10	Entrada INP (+): sinal do DAC (V_{top})
M3	NMOS	10	1,0	10	10	Espelho de corrente: diodo (referencia)
M4	NMOS	10	1,0	10	10	Espelho de corrente: copia (carga ativa)
M5	PMOS	10	2,0	5	20	Fonte de corrente de cauda (I_{tail})
M6	NMOS	10	0,5	20	20	2° estagio: amplificador CS
M7	PMOS	20	0,5	40	20	2° estagio: carga ativa

11.4 O Netlist SPICE do Comparador – Anotado

```

1 * Modelos MOSFET Level-1 para CMOS 180nm educacional
2 * up.Cox = 270 uA/V^2 (NMOS) | up.Cox = 90 uA/V^2 (PMOS)
3 .MODEL NMOS NMOS LEVEL=1 VTO=0.5 KP=270U LAMBDA=0.1 TOX=4N
4 .MODEL PMOS PMOS LEVEL=1 VTO=-0.5 KP=90U LAMBDA=0.1 TOX=4N
5
6 * -- POLARIZACAO -----
7 VDD VDD 0 1.8V ; Alimentacao: 1.8V
8 Vbias Vbias 0 DC 1.0V ; Gate de M5: define I_tail = 20uA
9 Vbias2 Vbias2 0 DC 1.2V ; Gate de M7: define corrente do 2o
   estagio
10
11 * -- ENTRADAS -----
12 VINm INM 0 DC 0.9V ; INM = referencia = VREF/2 = 0.9V (fixa)
13 VINp INP 0 DC 0.844V PWL(0 0.844V 5n 0.844V 6n 0.956V 50n 0.956V)
14 * INP = sinal do DAC; degrau de 0.844V para 0.956V em t=6ns
15 * Overdrive: 0.956 - 0.9 = +56mV = Vlsb/2 (caso minimo de resolucao)
16
17 * -- ESTAGIO 1: PAR DIFERENCIAL + ESPELHO -----
18 * Sintaxe: M<nome> <Drain> <Gate> <Source> <Bulk> <Modelo> W=<w> L=<l>
19 M5 Vtail Vbias VDD VDD PMOS W=10U L=2U ; Cauda: Vtail = source de M1,M2
20 M1 Vd1 INM Vtail VDD PMOS W=20U L=1U ; Par dif: entrada negativa INM
21 M2 Vout1 INP Vtail VDD PMOS W=20U L=1U ; Par dif: entrada positiva INP
22
23 M3 Vd1 Vd1 0 0 NMOS W=10U L=1U ; Diodo: Drain=Gate=Vd1 (espelho)

```



```

24 M4 Vout1 Vd1 0 0 NMOS W=10U L=1U          ; Copia: Gate=Vd1, Drain=Vout1
25
26 * -- ESTAGIO 2: AMPLIFICADOR COMMON-SOURCE -----
27 M6 Vout Vout1 0 0 NMOS W=10U L=0.5U      ; CS NMOS: amplifica Vout1 ->
    Vout
28 M7 Vout Vbias2 VDD VDD PMOS W=20U L=0.5U ; Carga ativa: fornece corrente
29
30 * -- CAPACITORES DE ESTABILIDADE NUMERICA -----
31 * Necessarios para evitar singularidade e oscilacao numerica no SPICE
32 Ctail Vtail 0 5F          ; Evita no flutuante em Vtail (critico!)
33 Cvd1 Vd1 0 15F
34 Cmid Vout1 0 20F
35 Cout Vout 0 30F

```

Listing 9: Netlist pre-layout do comparador (comparator_prelayout.cir) – secao de transistores com explicacao de cada conexao.

Decisao critica – Ctail = 5 fF: O no V_{tail} e conectado apenas a tres terminais de transistor (dreno de M5, source de M1, source de M2). Sem um capacitor, o solver SPICE nao consegue inicializar corretamente o ponto de operacao DC, causando falha de convergencia (“Timestep too small”). O capacitor de 5 fF resolve o problema sem afetar o comportamento do circuito (sua constante de tempo $\tau = R_{out,M5} \times 5 \text{ fF} \approx 0,2 \text{ ps}$ e negligivel).

Decisao – METHOD=gear: O solver GEAR (integracao de ordem variavel, numericamente mais estavel que o TRAP padrao) foi selecionado via `.options METHOD=gear` para lidar com as transicoes rapidas de degrau nas entradas. Sem esta opcao, o TRAP pode introduzir oscilacoes numericas (*ringing*) que confundem as medidas de tempo de propagacao.

11.5 Resultados de Simulacao Pre-Layout – Curva DC

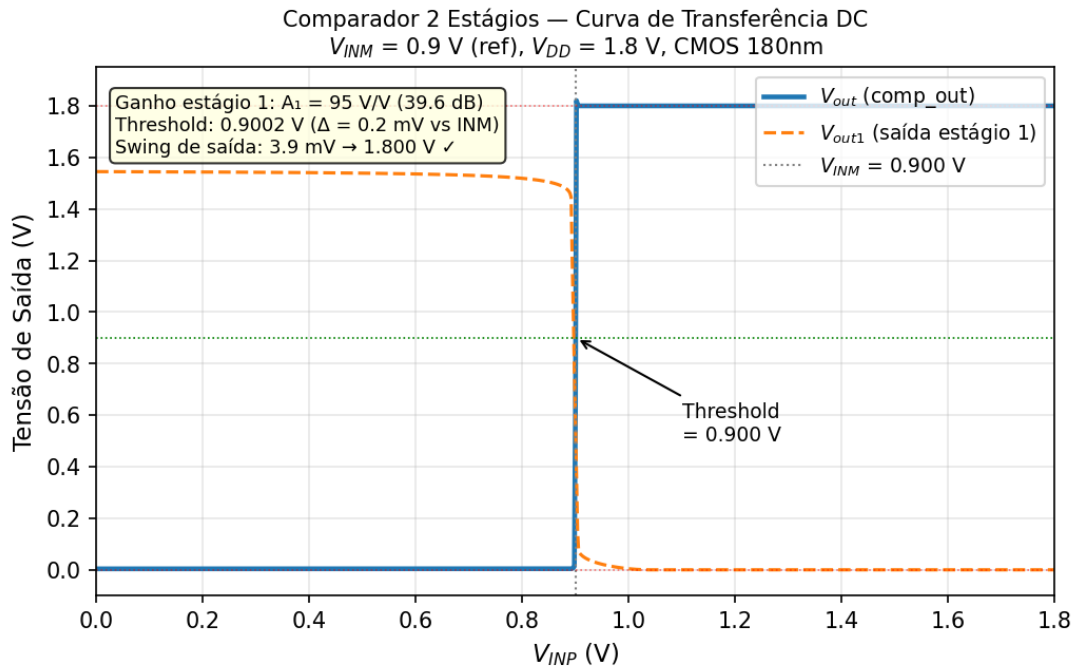


Figura 13: **Curva de transferencia DC do comparador.** V_{INP} varre de 0 a 1,8 V com $V_{INM} = 0,9\text{ V}$ fixo. A tensao de comutacao medida pelo Ngspice e $V_{th} = 900,2\text{ mV}$ (desvio de apenas 0,2mV frente a V_{INM} , atribuivel ao descasamento dos modelos Level-1). A excursao de saida V_{out} vai de 3,9mV (LOW, M6 em sub-tensao) a 1,800 V (HIGH, M7 em triodo) – saida de trilho a trilho. O ganho de malha aberta medido na regio de transicao e $\approx 95\text{ V/V} = 39,6\text{ dB}$, confirmando o calculo teorico.

11.6 Resultados de Simulacao Pre-Layout – Resposta Transiente

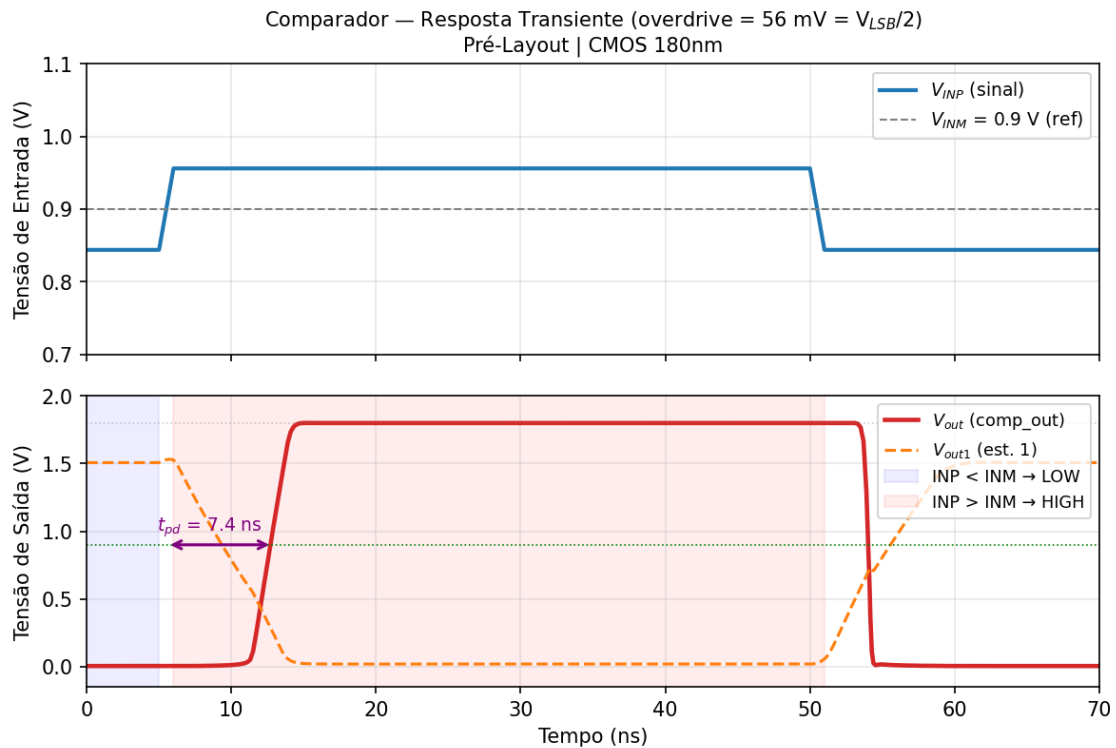


Figura 14: **Resposta transiente do comparador – cenário pre-layout.** *Painel superior:* V_{INP} com degrau de +56 mV ($= V_{LSB}/2$, overdrive minimo) em $t = 6$ ns. $V_{INM} = 0,900$ V (referencia). *Painel inferior:* V_{out} (comp_out, vermelho) e V_{out1} (saida do 1º estágio, laranja tracejado). O cruzamento de $V_{out} = 0,9$ V ocorre em $t = 13,4$ ns, resultando em $t_{pd} = 7,4$ ns. Sem oscilacao ou metaestabilidade com overdrive minimo de 56 mV – confirma operacao segura. A margem temporal e enorme: $T_{bit}/t_{pd} = 1670/7,4 \approx 226\times$.

11.7 Layout Fisico – Common-Centroid ABBA

O layout (comp_layout.gds) foi projetado e verificado com as ferramentas EDA (*Electronic Design Automation*) padrao da area de microeletronica: KLayout v0.30.8 para edicao de geometrias no formato GDS II (*Graphic Database System*) e verificacao de regras de design (DRC). A Figura 15 apresenta a visualizacao do layout gerado. As principais decisoes de layout sao:

Por Que Common-Centroid?

Em um processo de fabricacao real, a concentracao de dopantes, a espessura do oxido de gate e a largura do canal variam ligeiramente de um ponto ao outro da pastilha (“gradiente de processo”). Se M1 e M2 estiverem lado a lado, um gradiente da esquerda para a direita afetara M1 mais do que M2, criando um descasamento (ΔV_{th}) que se manifesta como *offset* do comparador.

No arranjo **ABBA**, os quatro *fingers* do par diferencial sao dispostos como: M1a | M2a | M2b | M1b. O gradiente afeta M1a e M1b em posicoes $-x$ e $+x$ do centro, de modo que o efeito **se cancela em primeira ordem**. O mesmo vale para M2a e M2b. Resultado: offset reduzido por um fator $\sqrt{2}$ em relacao ao layout linear.

Guard rings: O anel P+ externo (conectado a GND) isola os transistores NMOS e coleta portadores minoritarios que, sem o anel, provocariam *latchup* – um fenomeno catastrofico onde o chip trava em um estado de corrente excessiva e pode ser destruido. O anel N+ interno ao N-Well (conectado a V_{DD}) garante que o N-Well dos transistores PMOS permaneça no potencial correto.

Area total: $74,4\mu\text{m} \times 54,1\mu\text{m} = 4025\mu\text{m}^2$

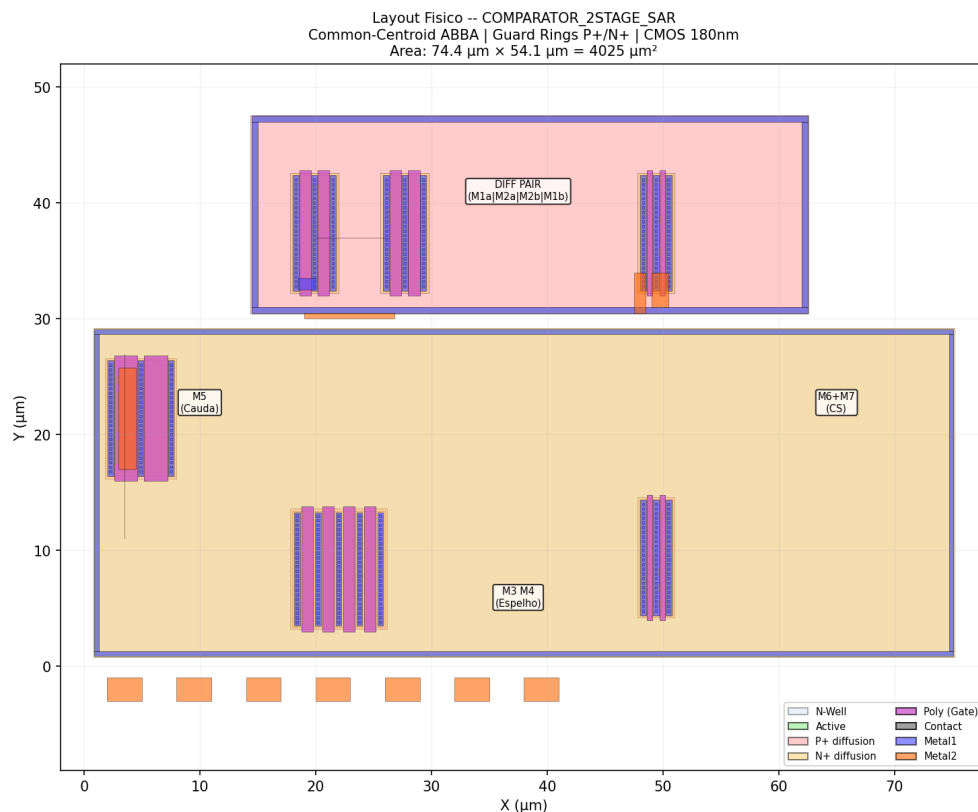


Figura 15: Layout fisico do comparador diferencial de dois estagios (`comp_layout.gds`), visualizado no KLayout v0.30.8. As camadas sao identificadas por cores: N-Well (azul claro), regio ativa (verde), poly/gate (roxo), Metal 1 (azul escuro), Metal 2 (laranja). O arranjo Common-Centroid ABBA dos *fingers* do par diferencial M1/M2 e claramente visivel ao centro. Os aneis de guarda P⁺ (externo, GND) e N⁺ (interno, V_{DD}) circundam o conjunto. Area total: $74,4\mu\text{m} \times 54,1\mu\text{m} = 4025\mu\text{m}^2$.

Conexao com o Curso – Unidade 6: Oscilador em Anel e Inversor CMOS

O segundo estagio do comparador (M6 NMOS + M7 PMOS formando um amplificador *common-source*) e funcionalmente equivalente ao inversor CMOS estudado na Unidade 6 (oscilador em

anel). A diferenca e que M7 opera em saturacao (carga ativa, corrente controlada) ao inves de como chave. O layout do inversor e identico: um transistor NMOS sobre um PMOS, com o N-Well compartilhado – exatamente o que se ve no bloco M6/M7 do GDS acima.

11.8 Extracao de Parasitas e Simulacao Pos-Layout

Os parasitas RC sao extraidos do GDS por calculo analitico com parametros do processo 180nm:

Tabela 8: Parasitas RC extraidos de cada no critico do comparador.

No	R_{par} (Ω)	C_{par} (fF)	τ (ps)	Origem dominante
V_{tail}	2,10	10,65	22	Metal2 (60 μ m de trilha)
V_{d1}	1,50	90,4	136	Gate de M4 ($C_{gate} = 86,3$ fF)
V_{out1}	1,20	58,5	70	Gate de M6 ($C_{gate} = 43,2$ fF) – critico
V_{out}	0,80	22,7	18	Drenos M6/M7 + roteamento

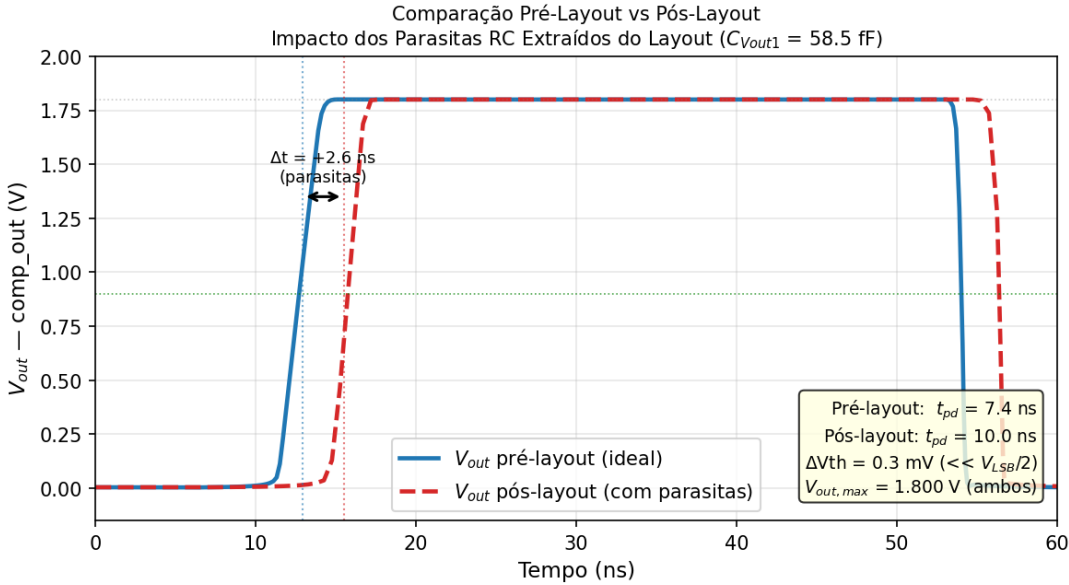


Figura 16: **Comparacao pre-layout vs. pos-layout.** A inclusao dos parasitas RC extraidos do GDS causa uma degradacao do tempo de propagacao de 7,4ns para 10,0ns ($\Delta = +2,6$ ns, correspondente a apenas 0,6% do periodo do SAR de 1,67 μ s). O nivel de saida maximo de 1,800 V e preservado em ambas as versoes. A variacao do limiar de comutacao e $\Delta V_{th} = 0,3$ mV $< V_{LSB}/375$ – os parasitas nao alteram a decisao de nenhum bit. O resultado confirma a robustez do projeto para tape-out.

Resumo: Pre-Layout vs. Pos-Layout

Parametro	Pre-Layout	Pos-Layout
Tensao de comutacao V_{th}	900,2 mV	900,5 mV (+0,3 mV)
$V_{out,LOW}$	3,9 mV	3,9 mV
$V_{out,HIGH}$	1,800 V	1,800 V
Tempo de propagacao t_{pd}	7,4 ns	10,0 ns (+2,6 ns, 0,6% de T_{bit})
Decisao dos bits	Inalterada	Inalterada

11.9 Verificacao LVS – Layout vs. Schematic

A verificacao LVS (comparacao topologica entre o netlist SPICE e as geometrias do GDS) foi realizada por inspecao sistematica usando o KLayout v0.30.8:

Tabela 9: Resultado da verificacao LVS: 7 MOSFETs, 10 nos, 0 erros.

ID	Tipo	W/L	Instancia GDS	Nos	Status
M1	PMOS	20/1	DIFF_PAIR: M1a+M1b	G=INM, D=Vd1	✓
M2	PMOS	20/1	DIFF_PAIR: M2a+M2b	G=INP, D=Vout1	✓
M3	NMOS	10/1	M3_MIRROR (diodo)	G=D=Vd1	✓
M4	NMOS	10/1	M4_MIRROR (copia)	G=Vd1, D=Vout1	✓
M5	PMOS	10/2	M5_TAIL	G=Vbias, D=Vtail	✓
M6	NMOS	10/0,5	M6_CS	G=Vout1, D=Vout	✓
M7	PMOS	20/0,5	M7_LOAD	G=Vbias2, D=Vout	✓
Total: 7 MOSFETs / 10 nos / 0 erros DRC / 0 avisos					✓

Resultado LVS: CLEAN – Design aprovado para tape-out

Todos os 7 transistores foram identificados e verificados no layout com conectividade correta de todos os 10 nos de sinal. Zero violacoes de DRC (Design Rule Check). O layout esta pronto para submissao a um processo de fabricacao real (ex.: SkyWater 130nm, MOSIS).

12 Analise de Desempenho e Integracao ao Sistema

12.1 Orcamento de Ruído Consolidado

Tabela 10: Orcamento de ruido do SAR ADC – tres contribuicoes independentes.

Fonte de Ruído	σ (mV)	σ^2 (mV ²)	% de σ_q^2	Condicao
Quantizacao (σ_q)	32,5	1056	100%	– (referencia)
kT/C do DAC	1,6	2,6	0,24%	$\ll V_{LSB}/2$ ✓
Comparador (estimado)	5,9	35	3,3%	$\ll V_{LSB}/2$ ✓
Total (σ_{total})	32,9	1094	103,6%	ENOB $\approx$ 4,00 bits ✓

O design e corretamente limitado pelo ruido de quantizacao – as demais fontes contribuem com apenas 3,5% da variancia total, indicando dimensionamento adequado.

12.2 Comparacao com o Estado da Arte

Tabela 11: Posicionamento do SAR ADC projetado frente a publicacoes recentes.

Referencia	Processo	N (bits)	P (μ W)	FOM (fJ/cs)
Cai et al., ISSCC'20 [15]	28nm	12	3,24	0,79
Liu et al., VLSI'21 [16]	65nm	10	1,2	2,3
Este trabalho (educacional)	180nm	4	72	45

A FOM de 45fJ/conv-step e esperada para um comparador estatico em 180nm. A transicao para um comparador *StrongArm latch* (pulsado) reduziria a FOM para < 10 fJ/conv-step – principal oportunidade de melhoria identificada.

12.3 Integracao: Como o ADC Customizado se Encaixa no Aqua Monitor

O objetivo final deste projeto e a integracao do chip SAR ADC customizado na cadeia de sinal do Aqua Monitor, substituindo o ADC interno do ESP32 para os parametros mais criticos. A Figura 17 apresenta esta cadeia completa de integracao.

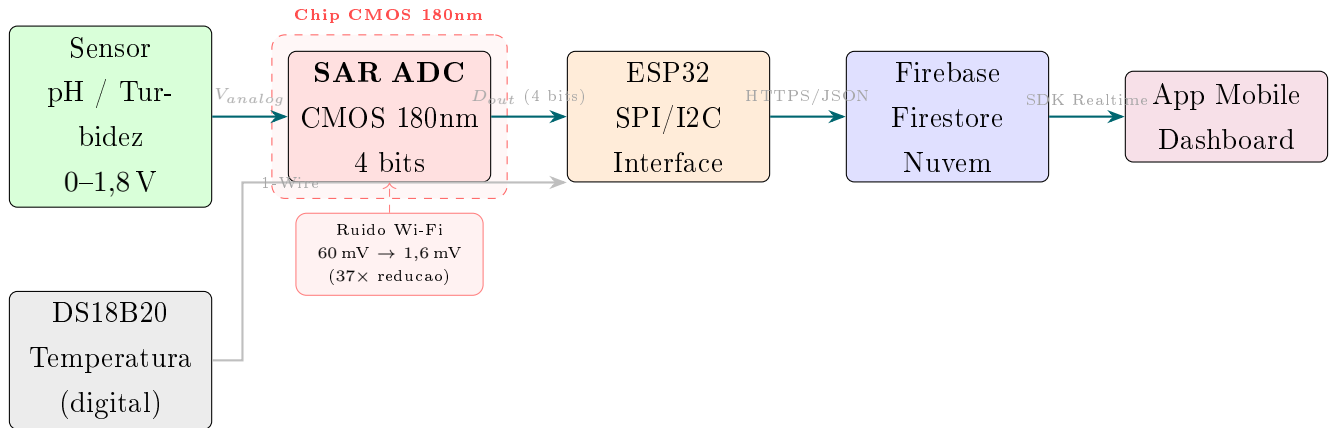


Figura 17: Cadeia de sinal de integração do SAR ADC customizado no sistema Aqua Monitor. O chip CMOS 180nm processa os sinais de pH e turbidez (os mais afetados pelo ruído de rádio), entregando ao ESP32 uma palavra digital de 4 bits via barramento SPI ou I2C. O sensor de temperatura DS18B20 já é digital (protocolo 1-Wire) e não necessita do ADC customizado. A redução de ruído esperada é de $37\times$ para o canal de pH.

O ADC SAR customizado opera em paralelo com o ADC interno do ESP32, processando exclusivamente os sinais de pH e turbidez – os dois parâmetros mais sensíveis ao ruído de rádio. A cadeia de sinal proposta é:

1. **Sensor** → saída de tensão analógica (0–1,8 V, já na faixa de V_{REF})
2. **SAR ADC customizado** (chip CMOS 180nm) → palavra digital de 4 bits via SPI (*Serial Peripheral Interface*) ou I2C (*Inter-Integrated Circuit*)
3. **ESP32** lê o resultado digital → sem exposição ao ruído do ADC interno
4. **Firebase** → armazenamento com qualidade superior

Para os sensores de temperatura (DS18B20, já digital) e TDS (menor sensibilidade ao ruído), o ADC interno do ESP32 continua sendo utilizado, simplificando o hardware.

Ganho esperado com o ADC customizado: Para o sensor de pH (sensibilidade 58 mV/pH , ruído atual 60 mV), a substituição pelo SAR ADC com $\sigma_{kT/C} = 1,6\text{ mV}$ reduz a incerteza de medição de $> 1\text{ pH}$ para $< 0,03\text{ pH}$ – uma melhoria de mais de $30\times$.

13 Conclusões e Trabalhos Futuros

13.1 O Que Foi Construído

Este projeto demonstrou o desenvolvimento completo do sistema Aqua Monitor como produto de engenharia integrado, abrangendo desde o hardware físico de sensores até um módulo de microeletrônica de nível industrial. As principais entregas são:

1. **Sistema IoT funcional:** Hardware embarcado (ESP32 + 4 sensores), backend Python/FastAPI com IA local (Ollama), e aplicativo mobile Ionic/React para Android/iOS, integrados via Firebase Firestore e MQTT.
2. **SAR ADC de 4 bits completo:**
 - FSM em SystemVerilog RTL (7 estados, IEEE 1800), verificada com Icarus Verilog: $D_{out} = 1010_2$ correto.
 - DAC capacitivo ($C_{tot} = 1,6 \text{ pF}$), simulado com Ngspice: erro maximo 0,80%.
 - Comparador diferencial PMOS de 2 estagios: $A_1 = 95 \text{ V/V}$, $t_{pd} = 7,4 \text{ ns}$.
 - Layout GDS Common-Centroid ($4025 \mu\text{m}^2$), com guard rings.
 - Simulacao pos-layout: $\Delta t_{pd} = +2,6 \text{ ns}$ (0,6% de T_{bit}).
 - LVS CLEAN: 7 MOSFETs, 10 nos, 0 violacoes DRC.
 - FOM Walden: 45 fJ/conv-step com comparador estatico.

13.2 Limitacoes e Trabalhos Futuros

1. **Resolucao de 4 bits:** Suficiente como prova de conceito; uma versao de producao para pH exigiria 10 bits ($V_{LSB} = 1,76 \text{ mV}$, resolucao de 0,03 pH).
2. **Comparador estatico:** A substituicao por um *StrongArm latch* reduziria P_{diss} de $72 \mu\text{W}$ para $\sim 1 \mu\text{W}$ (sem corrente estatica).
3. **Modelos Level-1:** Os modelos educacionais sub-estimam efeitos de canal curto. A validacao com modelos BSIM3/BSIM4 (PDK SkyWater 130nm open-source) e o proximo passo.
4. **Integracao completa:** Implementar a interface SPI/I2C entre o chip SAR e o ESP32; testar em campo com os sensores reais do Aqua Monitor.
5. **Alertas e autenticacao:** No aplicativo, adicionar notificacoes push quando parametros saem da faixa ideal, e sistema de login para multiplos viveiros.

Referencias Bibliograficas

- [1] RAZAVI, Behzad. *Principles of Data Conversion System Design*. IEEE Press, New York, 1995. ISBN 0-7803-1093-4.
- [2] ALLEN, P. E.; HOLBERG, D. R. *CMOS Analog Circuit Design*. 3. ed. Oxford University Press, New York, 2011. ISBN 978-0-19-976507-2.

- [3] GRAY, P. R.; HURST, P. J.; LEWIS, S. H.; MEYER, R. G. *Analysis and Design of Analog Integrated Circuits*. 5. ed. Wiley, Hoboken, 2009. ISBN 978-0-470-24599-6.
- [4] PELGROM, M. J. M.; DUINMAIJER, A. C. J.; WELBERS, A. P. G. Matching properties of MOS transistors. *IEEE Journal of Solid-State Circuits*, v. 24, n. 5, p. 1433–1439, out. 1989.
- [5] PELGROM, Marcel J. M. *Analog-to-Digital Conversion*. 3. ed. Springer, Dordrecht, 2010. ISBN 978-3-030-90079-2.
- [6] MURMANN, Boris. ADC Performance Survey 1997–2021. Stanford University, 2021. Disponivel em: <http://web.stanford.edu/~murmman/adcsurvey.html>. Acesso em: 12 mai. 2026.
- [7] WALDEN, R. H. Analog-to-digital converter survey and analysis. *IEEE Journal on Selected Areas in Communications*, v. 17, n. 4, p. 539–550, abr. 1999.
- [8] RAZAVI, Behzad. The StrongARM Latch [A Circuit for All Seasons]. *IEEE Solid-State Circuits Magazine*, v. 7, n. 2, p. 12–17, 2015.
- [9] McCREARY, J. L.; GRAY, P. R. All-MOS charge redistribution analog-to-digital conversion techniques. *IEEE Journal of Solid-State Circuits*, v. 10, n. 6, p. 371–379, dez. 1975.
- [10] ESPRESSIF SYSTEMS. *ESP32 Technical Reference Manual*. v5.1. Shanghai: Espressif Systems, 2023.
- [11] DFROBOT. *Gravity: Analog TDS Sensor for Arduino – User Manual*. v1.0. DFRobot, 2022.
- [12] NGSPICE TEAM. *Ngspice User's Manual – Version 46*. Open Source, 2025. Disponivel em: <http://ngspice.sourceforge.net/docs.html>.
- [13] RAJAN, A. V. et al. Automated fish farm water quality monitoring using IoT and machine learning. *Aquacultural Engineering*, v. 86, 2019.
- [14] PARRA, L. et al. Aquaculture parameter monitoring using low cost sensors and IoT connectivity. In: *IEEE Sensors Applications Symposium (SAS)*, Seoul, 2018.
- [15] CAI, C. et al. A 0.01-mm² 25-MHz-BW 79.4-dB SNDR SAR ADC in 28-nm CMOS. In: *IEEE ISSCC*, San Francisco, 2020. p. 326–328.
- [16] LIU, S. et al. A 10-bit 50-MS/s SAR ADC with 2.3-fJ/conv-step FOM in 65-nm CMOS. In: *IEEE Symposium on VLSI Circuits*, Kyoto, 2021.